



The Open University

M263 Building Blocks of Software

Unit 11

Databases

Study guide

Sections 2 and 4 are longer than the other sections in this unit. Both contain important ideas, so allow more time for studying these two sections. Note that the exercises on Section 4 cover ideas from the whole unit and are longer as a result. There is no practical computing associated with this unit.

This publication forms part of an Open University course. Details of this and other Open University courses can be obtained from the Course Information and Advice Centre, PO Box 724, The Open University, Milton Keynes, MK7 6ZS, United Kingdom: tel. +44 (0)1908 653231, e-mail general-enquiries@open.ac.uk

Alternatively, you may visit the Open University website at <http://www.open.ac.uk> where you can learn more about the wide range of courses and packs offered at all levels by the Open University.

To purchase a selection of Open University course materials, please visit the Open University webshop at www.ouw.co.uk, or contact Open University Worldwide, Michael Young Building, Walton Hall, Milton Keynes MK7 6AA, United Kingdom for a brochure. tel. +44 (0)1908 858785; fax +44 (0)1908 858787; e-mail ouwengq@open.ac.uk

The Open University, Walton Hall, Milton Keynes, MK7 6AA.

First published 2004. Second edition 2008.

Copyright © 2008 The Open University

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, transmitted or utilised in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without written permission from the publisher or a licence from the Copyright Licensing Agency Ltd. Details of such licences (for reprographic reproduction) may be obtained from the Copyright Licensing Agency Ltd of 90 Tottenham Court Road, London W1T 4LP.

Edited, designed and typeset by The Open University, using the Open University T_EX System.

Printed and bound in the United Kingdom by The Charlesworth Group, Wakefield.

ISBN 978 07492 2303 8

Contents

Introduction	4
1 Tables and relations	5
2 Functional dependency and keys	10
2.1 Functional dependency	11
2.2 Implicit dependencies	14
2.3 Keys	22
3 Linking schemes by foreign keys	25
4 Normal forms	31
4.1 Second normal form	32
4.2 Third normal form	36
Summary of Unit 11	42
Solutions to the exercises	43
Index	48

Introduction

Databases are one of the key applications in modern computing. The need for storing large amounts of information in such a way that it can be accessed easily is central to much commercial and government activity. In this unit we will consider the form of databases known as relational databases. These are one of the main forms of information storage. We shall only be concerned here with one aspect of such databases, but one that is of central importance when designing a relational database.

In the first unit of the course we discussed the type of data needed for stock control in a supermarket and some of the processes carried out when a customer is at a till. This is a useful example to consider. It involves a large amount of data of many different types, some linked and some not linked. For example, there is data on the stock; there is data on the staff; and there is data on the customers. In addition, this data is changing all the time. The system may be required to keep track of stock for ordering purposes. It may also need to keep track of which staff have been operating tills, in case of errors or complaints. It may need to analyse spending patterns of customers who hold store cards — perhaps to target “special offer” vouchers to be sent to them. To design and set up such a database is a lengthy process and has several stages. In the first of these, the client and design team discuss what data needs to be recorded and the type of information that should be easily extracted from the stored data. However, we shall not be concerned here with this stage of database design. Nor will we cover obtaining information from a database. These topics require more time than we have available, and are better covered in a specialist database course.

To provide an overview of the ideas that are covered here, we need to look at an example in some detail. The fundamental form of data storage used in a relational database can be pictured as a table. For example, information about students and their tutors might be stored as below.

<i>courseCode</i>	<i>studentID</i>	<i>tutorID</i>	<i>studentEmail</i>	<i>tutorEmail</i>
M263	A1234	0123	qjh3@st.ac.uk	tmj2@tu.ac.uk
M263	A1245	0124	qjh3@st.ac.uk	ckj2@tu.ac.uk
M263	A1257	0123	abc3@st.ac.uk	tmj2@tu.ac.uk

In such a table, the columns have headings that indicate what feature of the data is stored in them. Each row is an actual piece of data. The headings of the table are known as the **attributes** of the data. The choice of attributes and the way that they are grouped in tables form crucial features of the design of a relational database. A well considered design can make the database easier and more reliable to use. For example, in the table above, the tutor with *tutorID* 0123 may occur in several entries in the table. Suppose that this tutor changes his email address. It would then be necessary to locate every entry relating to that tutor, and correct the entry in the *tutorEmail* column.

Suppose that, instead, we use three tables, with the following columns.

courseCode, *studentID*, *tutorID*.
tutorID, *tutorEmail*.
studentID, *studentEmail*.

In this case, the first table gives details of which tutor a student has for each course, the second table gives the email address of each tutor, and the third

This is called a **requirements analysis**.

This is done using a query language.

Such a table provides a way of visualising how the data is structured — an abstraction. We will not discuss the details of how such a table might be implemented.

The word attribute is also used in the context of a class implementation. Here, we are using the word in a different context. The two uses of the term are related — but we shall not discuss this here.

table gives the email address of each student. In this case, there will be a single entry — in the second table — giving a tutor’s email address, and so just a single entry to locate and correct if the email address is changed.

For such a design to work, we need the tables to be connected. Such a group of linked tables forms a relational database, and we shall consider in Section 3 how we can associate tables. In Section 4, we shall discuss how we can examine a proposed design for a table and recognise when updating problems may occur — and how we might then determine an appropriate way of dividing up the attributes into connected tables to avoid such problems.

In Sections 1 and 2 we will introduce some fundamental ideas concerning such tables of data. A **relation scheme** is a set of attributes (column headings), together with logical connections between them. Such a logical connection is known formally as a **functional dependency**. Recognition of functional dependencies between attributes is the basis for understanding and analysing the structure of a relation scheme. Another fundamental idea is that of a **key** of a scheme, which is an attribute or set of attributes that picks out a unique row in a table. For example, consider a table with attributes *courseCode*, *studentID*, *tutorID* that gives the courses studied by Open University students in 2004, and their tutor for each course (and assume that a student has only one tutor for a particular course). Then {*courseCode*, *studentID*} will form a key for the relation scheme {*courseCode*, *studentID*, *tutorID*}. We shall discuss how we can identify possible keys of a scheme in Section 2.

Since there will be at most one row in the table with particular values of both *courseCode* and *studentID*.

1 Tables and relations

In this section, we shall introduce some fundamental concepts and terminology concerning relational databases. For example, students’ records on a database at a university might be stored as below.

Student Table				
<i>studentID</i>	<i>name</i>	<i>postcode</i>	<i>email</i>	<i>current</i>
“M1234”	“Hamid”	“S10 3AB”	“qjh3@st.ac.uk”	true
“M2345”	“Jones”	“LS21 4CD”	“wj256@st.ac.uk”	false

In this table, no two students will have the same *studentID*. The table records a student’s name, postcode of their current address, an email address, and whether or not they are currently studying with the university (*current* shows **true** if they are). As noted above, the column headings are called **attributes**, and show what feature of a data item is stored in that column. A row is called a **tuple**. Each tuple gives a data item — an entry in the database. A collection of tuples forming a complete table is a **relation**. We need to examine these concepts a little more carefully.

Pronounce “tuple” to rhyme with “couple”.

We first consider the attributes. One would not want to have a table with two columns headed *name* for example. No attribute should be repeated in the headings. Formally, the column headings form a *set* of attributes. However, when giving a table forming a relation, it is convenient always to write the attributes in the scheme in the same order. We can then give a particular tuple in the following form.

(“M1234”, “Hamid”, “S10 3AB”, “qjh3@st.ac.uk”, **true**)

Such a tuple consists of a value for each of the attributes. Giving the attributes in a fixed order ensures that we know which value relates to which attribute. For example, in the tuple above, the second entry gives the value of the second attribute — *name* — the name of the student.

You may recall that we introduced the terminology “tuple” in Unit 1. We are using it in the same way here. A tuple is a member of the Cartesian product of the data sets corresponding to the attributes. We turn to these sets now.

See Unit 1.

Each attribute has an associated data set called the **domain** of the attribute. For example, *current* is associated with the data set *Bool*. All entries in the column headed *current* should belong to the set $\{\text{true}, \text{false}\}$. The domain is the set of values that an attribute may take. This may consist of the whole of the data set of some standard type — or it may be that a data entry must take a particular form. Suppose, for example, that the *studentID* is given as a string, but only strings of a particular form are permitted — say, a string containing five characters, the first of which is an upper-case letter, and the other four of which are digit characters. Then we can see that “L5432” is a suitable value for *studentID*, but “XX34y” is not. We could then define precisely the domain of *studentID*, say *StudentIDCode*, as a set of strings satisfying these conditions.

It is helpful if each attribute has a distinct domain. In that case, we can readily tell when, for example, a name has been entered where a student ID should have been. However, this cannot always be achieved. For example, suppose that there are attributes giving a daytime and an evening telephone number. These will come from the same set, so these attributes will have the same domain. Suitable choice of domains is not an issue that will concern us here. We shall give a domain for each attribute. This will often be a set drawn from a standard type in a way that is not spelt out in detail. For example, we described above a possible domain set *StudentIDCode* for the attribute *studentID*, but we shall not give such details in future. (Where a domain contains just a few acceptable values, then we will give it in full.)

We will also give a data type for each attribute. In practice, one would want to be able to manipulate information extracted from a database, and we would then need to know the data type of an attribute to recognise what operations were available to perform such manipulations. However, we shall not be concerned with data manipulation here. The type of an attribute might be a standard (library) class, or it might be a user-defined class tailored to the application. We will not discuss the design or use of user-defined classes, nor will we expect you to make decisions concerning suitable domains or types for attributes. We may assume that some attribute has a type that is a user-defined class. As with domains, we will not give details of such classes.

One might also want to manipulate the database itself, for example to sort tuples in a table before displaying them on screen.

These are not needed for the issues discussed in this unit.

Mathematically, we can say that a relation scheme is a set of attributes, and that a relation over the relation scheme is a set of tuples, where each tuple comes from the Cartesian product of the data sets of the attributes. However, there is one important factor missing from this description.

A chief characteristic of databases is that the data changes. This gives rise to a subtle problem. A relation is represented by an actual table of data. If we add a row to this table we change the relation. In this unit, we shall discuss properties of such a relation that do not change every time we add or delete an entry. These properties will depend on the attributes and on any underlying rules that apply to entries to a given table.

In the Student Table example, the attribute *studentID* determines a unique student (a person in the real world). Assume that a student is asked to supply a single address and a single email address. Then it would not be appropriate to have two tuples in a table with the same value of *studentID* and different values of *email*. Similarly, it would not be appropriate to have two tuples in a table with the same value of *studentID* and different values of *postcode*. Such rules about acceptable data are an integral part of the design of a relation scheme. So we shall use the term **relation scheme** to mean a set of attributes *together with* any underlying rules that apply. A relation over the relation scheme must obey these underlying rules.

We have defined a relation as a set of tuples. Consequently, the order in which the tuples are given in a table is not significant. Change the order of the rows in the table, and we still have the same relation. Since the attributes are also a set, we would also still have the same relation if we changed the order of the columns in a table. However, as noted above, we will not do this, because to do so would make it harder to keep track of which item in a tuple relates to each attribute. If the attributes are given in a fixed order, the tuple (“M2345”, “Jones”, “LS21 4CD”, “wj256@st.ac.uk”, **false**) can be interpreted.

The format we will use for describing relation schemes is exemplified below. In tables, the attributes will be given in the order in which they are listed in the scheme.

We might well care about the order of the rows when we display them on a screen or print them, but that is a separate issue.

STUDENT			Relation scheme
Attribute	Domain	Type	
<i>studentID</i>	<i>StudentIDCode</i>	mString	
<i>name</i>	<i>Name</i>	mString	
<i>postcode</i>	<i>Postcode</i>	mString	
<i>email</i>	<i>email</i>	mString	
<i>current</i>	<i>Bool</i>	Bool	
<i>Rules</i>			
R1 Two students cannot share the same <i>studentID</i> .			
R2 A student must give a single <i>name</i> , a single <i>postcode</i> and a single <i>email</i> address, and has a single <i>current</i> status.			

Activity 1.1

Which of the following tuples could be included in a relation over the scheme **STUDENT**?

- (a) (“H5520”, “Norman”, “NN55 1LL”, “qqn@st.ac.uk”, 0)
- (b) (“H5520”, “Norman”, “NN55 1LL”, “qqn@st.ac.uk”, **false**)
- (c) (“H5520”, “Norman”, “NN55 1LL”, “qqn@st.ac.uk”)

Discussion The tuple in (c) only contains four values. Since there are five attributes in the scheme, this could not appear in a relation. Nor could the tuple in (a), because the last value is of the wrong type; 0 is not of type **Bool**. The only tuple that is suitable is that in (b).

Activity 1.2

In the relation scheme below, course codes are strings of length 4, where the first character is an upper-case letter and the remainder are digits.

COURSE		Relation scheme
Attribute	Domain	Type
<i>courseCode</i>	<i>CourseCode</i>	mString
<i>title</i>	<i>CourseTitle</i>	mString
<i>credit</i>	{10, 15, 20, 30, 60}	Int
<i>Rules</i>		
R1 Two courses cannot share the same course code.		
R2 A course has one title and one credit rating.		

The following table shows a relation Course Table over this scheme.

Course Table		
<i>courseCode</i>	<i>title</i>	<i>credit</i>
C123	Databases	30
C224	HCI	15
C421	AI	60

Strictly, values of attributes of type **mString** should be enclosed in quotes. For convenience, we will relax this rule throughout this unit. Confusion is unlikely in the current context.

- (a) Write down an element of the relation Course Table.
- (b) Which of the following tuples could be added to the relation Course Table?
 - (i) (C123, Software design, 60)
 - (ii) (C124, Databases, 60)
 - (iii) (C321, Software design, 100)
 - (iv) (C321, Software design, 60)

Discussion

- (a) Any of the three given tuples is an element: (C123, Databases, 30) or (C224, HCI, 15) or (C421, AI, 60).
- (b) Only (ii) and (iv) are allowable. (i) uses a value of *courseCode* that already exists and so to add it would break R2. (iii) has an incorrect number of credit points (the value 100 does not appear in the domain for *credit*). In (ii), the value of *courseTitle* is the same as that for course C123. You may feel that this should be invalid, but there is nothing in the given rules to preclude it.

Activity 1.3

A relation scheme called **CENTRE_TIMETABLE** and a relation over the scheme are given below.

CENTRE_TIMETABLE			Relation scheme
Attribute	Domain	Type	
<i>date</i>	<i>Date</i>	Date	
<i>tutor</i>	<i>TutorName</i>	mString	
<i>courseCode</i>	<i>CourseCode</i>	mString	
<i>room</i>	<i>CentreRoom</i>	mString	
<i>Rules</i>			
R1 On a given day, each room can be used at most once.			
R2 On a given day, a tutor can teach at most one course.			
R3 On a given day, a tutor can teach in at most one room.			

CentreTimetable Table

<i>date</i>	<i>tutor</i>	<i>courseCode</i>	<i>room</i>
March 1	Professor Plum	F123	Hlab
March 1	Dr Apple	F245	LR24
March 2	Ms Cherry	A104	LR21
March 8	Dr Apple	F245	LR24

Suggest a tuple that could be added to this relation without breaking the rules. Can you see any problem with this relation scheme?

Discussion There are many possible answers to this. One answer would be (March 8, Ms Cherry, A104, LR21). Using values that appear in the given table ensures that the values of the attributes belong to the appropriate domain (so long as they are given in the correct order). You need to ensure that the values you have for *tutor* and *room* do not appear in another tuple in the relation with the same value of *date*. For example, (March 8, Ms Cherry, A104, LR24) could not be added. (There would then be two tutors in the same room on March 8, which would break R1.)

A possible problem with this relation scheme is that it assumes that the name given to the tutor is unique; that is, no two tutors share the same name. There are two commonly used solutions to this problem. One is to have an identity code that is unique to each tutor. The other is to have a rule that no two tutors can share the same name (if necessary, in the entry under *tutor* the tutor’s name can be amended to ensure that it is unique — for example, by using “Mr J.Smith2”).

Objectives for Section 1

After studying this section you should be able to do the following.

- ☐ Use the terminology: attribute; tuple; relation; relation scheme.
- ☐ Understand that a relation is a set of tuples.
- ☐ Understand that a relation scheme is a set of attributes, together with rules that must be satisfied by a valid relation over the scheme.

Exercises on Section 1

Exercise 1.1

A relation scheme **BIRD** and a relation over that scheme are given below. The relation stores information about species of bird: whether or not they migrate to the UK to breed, and their typical choice of nest site.

BIRD			Relation scheme
Attribute	Domain	Type	
<i>commonName</i>	<i>BirdName</i>	mString	
<i>migrant</i>	<i>Bool</i>	Bool	
<i>nestSite</i>	<i>NestSite</i>	mString	
<i>Rules</i>			
R1 No two bird species share the same common name.			
R2 For each bird species there is only one description of nest site, and of whether they are a migrant.			

Bird Table		
<i>commonName</i>	<i>migrant</i>	<i>nestSite</i>
Dipper	false	river banks
Blue Tit	false	holes in trees
Swallow	true	barns
Pied Flycatcher	true	holes in trees
Manx Shearwater	true	burrows in the ground

- (a) Write down a tuple of the given relation.
- (b) Which two of the following tuples could not be added to the table?
- (i) (Magpie, **false**, trees)
 - (ii) (Puffin, don't know, burrows in the ground)
 - (iii) (Blue Tit, **false**, nestboxes)
 - (iv) (Skylark, **false**, on ground)
- What do you need to know to ensure that the other two tuples are valid entries?

2 Functional dependency and keys

In each of the relation schemes introduced in the last section, there were restrictions on the tuples that could appear together in a relation over the scheme. In the relation scheme **STUDENT**, no two tuples can have the same value of *studentID*. In the scheme **CENTRE_TIMETABLE**, one could not have two tuples with both *date* and *room* the same. In the introduction, we discussed a scheme in which two tuples with the same value for the attribute *tutor* would also have to have the same value of *tutorEmail*.

Identifying such rules is a necessary step in describing a relation scheme, as we noted in Section 1.

In this section, we shall look at rules of this type. The approach we use would make it possible for a computer to detect invalid modifications to a relation automatically. It is also the basis for our analysis of the structure of relation schemes later in this unit.

2.1 Functional dependency

In the relation scheme **STUDENT**, the value of *studentID* identifies an actual student. The rules of the scheme require that there can be only one value of each of the other attributes — *name*, *postcode*, etc. — associated with a particular student. In this situation we say that the other attributes are **functionally dependent** on $\{studentID\}$. We use the notation $\rightsquigarrow$ to indicate functional dependency, and write, for example,

$\{studentID\} \rightsquigarrow \{email\}$

to indicate that the attribute *email* is functionally dependent on the attribute *studentID*.

This set notation is easily extended to show that more than one attribute is dependent on *studentID*. Indeed, we have

$\{studentID\} \rightsquigarrow \{name, postcode, email, current\}$.

The rules of the scheme **STUDENT** mean that each of the other attributes is determined by *studentID*.

It is also possible to have several attributes on the left-hand side of a functional dependency. In the relation scheme **CENTRE_TIMETABLE**, for example, *room* is functionally dependent on the two attributes *date* and *tutor*. This is a consequence of rule R3 of the scheme, that a tutor can teach in at most one room on a given day. This rule means that, once you know the tutor and the day, there can be at most one corresponding room. Similarly, rule R2 of this scheme means that *courseCode* is functionally dependent on *date* and *tutor*. Given values for both *tutor* and *date*, we can have at most one corresponding value for *room* or for *courseCode*. In this case, we have the functional dependency

$\{date, tutor\} \rightsquigarrow \{room, courseCode\}$.

Now consider the relation scheme **STUDENT_TUTOR**, below. In this, *studentID* identifies a unique student (that is, two students cannot have the same *studentID*). Also, *tutorID* identifies a unique tutor.

STUDENT_TUTOR			Relation scheme
Attribute	Domain	Type	
<i>courseCode</i>	<i>CourseCode</i>	mString	
<i>studentID</i>	<i>StudentIDCode</i>	mString	
<i>tutorID</i>	<i>TutorIDCode</i>	mString	
<i>studentEmail</i>	<i>email</i>	mString	
<i>tutorEmail</i>	<i>email</i>	mString	
Rules			
R1 A tutor must have exactly one email address.			
R2 A student must have exactly one email address.			
R3 A student has only one tutor for a given course.			

Such rules may be an inevitable consequence of the data being stored, or a choice made as part of the process of database design.

See page 7.

Read $\rightsquigarrow$ as “determines”. It can be typed as $\rightsquigarrow$.

See page 9.

Professor Plum can have at most one tutorial on March 1, for example, which will involve only one course and one room.

It will often be the case that an attribute is used as a unique identifier for some entity in the “real world”. We will usually not give a separate rule for the scheme stating this.

The value of *tutorID* identifies a unique tutor, and then R1 states that the tutor must have only one email address. So whenever a particular value of *tutorID* occurs in a tuple, it must occur with the same value of *tutorEmail*. We have a functional dependency

$$\{tutorID\} \rightsquigarrow \{tutorEmail\}.$$

These examples enable us to formulate a definition of what we mean by functional dependency.

Let A be a set of attributes, and x be another attribute of the scheme. Suppose that the following holds.

In a relation over the scheme, if two tuples have the same value for all the attributes in A , then they must also have the same value of x .

We then say that x is **functionally dependent** on the set of attributes A , and write $A \rightsquigarrow \{x\}$.

Note that functional dependencies are consequences of the rules of a relation scheme.

More generally, if B is a set of attributes, we write $A \rightsquigarrow B$ if $A \rightsquigarrow \{x\}$ for every attribute x in the set B .

It is not always easy to work with a formal definition such as this, but it does give a concrete test for recognising functional dependencies — and when they do not occur. If we can have two tuples with the same values of an attribute y and different values of an attribute x , then x is not determined by y . To state that such a functional dependency does not hold, we write $\{y\} \not\rightsquigarrow \{x\}$. For example, a student may be doing more than one course. A relation over **STUDENT_TUTOR** can contain tuples with the same value of *studentID* and different values of *courseCode*. So $\{studentID\} \not\rightsquigarrow \{courseCode\}$.

You can type $\not\rightsquigarrow$ as $/\sim>$.

Functional dependencies arise from the rules describing a relation scheme. To determine what functional dependencies follow from the rules of a scheme, it can be useful to think in the following terms, exemplified by reference to **STUDENT_TUTOR**. “If I know the tutor, then I know their email address.” That perception allows you to see that the dependency $\{tutorID\} \rightsquigarrow \{tutorEmail\}$ holds. Similarly, in **STUDENT**, if you know the student, then you know their name, postcode, email and current status.

EXAMPLE 2.1

We will investigate the functional dependencies that must be satisfied by the relation scheme **BUS**, below. Here *busCode*, *driverID* and *routeCode* are unique identifiers for the bus, driver and route.

BUS			Relation scheme
Attribute	Domain	Type	
<i>busCode</i>	<i>BusCode</i>	mString	
<i>driverID</i>	<i>DriverID</i>	mString	
<i>routeCode</i>	<i>RouteCode</i>	mString	
<i>day</i>	<i>Day</i>	mString	
Rules			
R1 A bus has only one driver on a given day.			
R2 A driver can only drive one bus on a given day.			
R3 A bus can only cover one route on a given day.			

Day is the set {Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday}. Below, we give part of a relation over the scheme. This helps to illustrate the rules. (We assume that the allocation of drivers to buses and buses to routes is the same during each week throughout the period for which the schedule shown in the relation is applicable.)

	<i>busCode</i>	<i>driverID</i>	<i>routeCode</i>	<i>day</i>
t1	VV1	d12	R1	Monday
t2	GN1	d13	R2	Monday
t3	VV2	d12	R1	Tuesday
t4	VV1	d13	R3	Tuesday
t5	GN1	d14	R1	Tuesday
t6	VV1	d13	R1	Wednesday
t7	VV2	d12	R1	Wednesday
⋮	⋮	⋮	⋮	⋮

The first column gives labels for the tuples that we can use for reference. They are not part of the relation.

We show the bottom of the relation like this to indicate that it is not complete.

The first rule, R1, means that if we know the bus and the day, then we know the driver. This gives

FD1: $\{busCode, day\} \rightsquigarrow \{driverID\}$.

R2 means that if we know the driver and day then we know the bus, i.e.

FD2: $\{driverID, day\} \rightsquigarrow \{busCode\}$.

R3 means that if we know the bus and day then we know the route, i.e.

FD3: $\{busCode, day\} \rightsquigarrow \{routeCode\}$.

Now if we know the bus and the route, we would not expect this to determine the driver. Different drivers might drive the same bus on the same route on different days. The best way to show that a functional dependency does not hold is to produce tuples that show that it does not hold. In the given table, we have the tuples t1 and t6 that have the same bus and route but different drivers. This shows that $\{busCode, routeCode\} \not\rightsquigarrow \{driverID\}$ — this functional dependency does not hold.

Identifying functional dependencies means that we can decide which tuples could be added to a relation. For example, we cannot add the tuple (GW2, d12, R3, Wednesday) to the table above since this, in combination with t7, would violate the dependency FD2.

Activity 2.1

Identify the functional dependencies that follow from the rules given earlier for the relation scheme **STUDENT_TUTOR**.

Discussion

If we know the tutor then we know the tutor’s email address (R1).

FD1: $\{tutorID\} \rightsquigarrow \{tutorEmail\}$.

If we know the student then we know the student’s email address (R2).

FD2: $\{studentID\} \rightsquigarrow \{studentEmail\}$.

If we know the student and the course then we know the tutor (R3).

FD3: $\{studentID, courseCode\} \rightsquigarrow \{tutorID\}$.

2.2 Implicit dependencies

The rules of a relation scheme lead directly to certain functional dependencies. As well as these, there will be other functional dependencies satisfied by the scheme. For example, whatever attribute x may be, we will have $\{x\} \rightsquigarrow \{x\}$. (“If I know the value of x then I know the value of x .”) There are other ways of deriving further functional dependencies that are not quite so obvious, but correspond to equally commonsense ideas. For example, consider FD1 and FD3 in the scheme **BUS**.

See Example 2.1.

FD1: $\{busCode, day\} \rightsquigarrow \{driverID\}$.

FD3: $\{busCode, day\} \rightsquigarrow \{routeCode\}$.

FD1 says that if we know the values of $busCode$ and day then we know the value of $driverID$, while FD3 says that if we know the values of $busCode$ and day we know the value of $routeCode$. From these, we can deduce a new functional dependency:

$\{busCode, day\} \rightsquigarrow \{driverID, routeCode\}$.

This example illustrates a general rule for sets of attributes X, A, B , which we can express formally as follows.

if $X \rightsquigarrow A$ and $X \rightsquigarrow B$ then $X \rightsquigarrow A \cup B$.

If a set of attributes, X , determines more than one thing, then we can combine all the things X determines into a single set.

If we have the same thing on the left, we can combine the things on the right.

We can reverse this process. Starting with the functional dependency

$\{busCode, day\} \rightsquigarrow \{driverID, routeCode\}$,

we could deduce from it either of the two functional dependencies

FD1: $\{busCode, day\} \rightsquigarrow \{driverID\}$,

FD3: $\{busCode, day\} \rightsquigarrow \{routeCode\}$.

Another general rule for sets of attributes X, A, C is:

if $X \rightsquigarrow A$ and $C \subseteq A$ then $X \rightsquigarrow C$.

We can replace the set on the right by a subset.

Here is a collection of formal rules that can be used to deduce dependencies.

Result 2.1

If X, A, B and C are sets of attributes of the same scheme, then

1. $X \rightsquigarrow X$.
2. If $X \rightsquigarrow A$ and $X \rightsquigarrow B$ then $X \rightsquigarrow A \cup B$.
3. If $X \rightsquigarrow A$ and $C \subseteq A$ then $X \rightsquigarrow C$.
4. If $X \rightsquigarrow A$ and $X \subseteq C$ then $C \rightsquigarrow A$.
5. If $A \rightsquigarrow B$ and $B \rightsquigarrow C$ then $A \rightsquigarrow C$.

Rule 4 above says that you can replace the set on the left of a dependency by something bigger. For example, in **BUS**, $\{driverID, day\} \rightsquigarrow \{busCode\}$. If knowing the driver and the day tells us the bus, then knowing the driver, the day and the route would still tell us the bus. (The extra information of knowing the value of $routeCode$ does not change the value of $busCode$.) So $\{driverID, day, routeCode\} \rightsquigarrow \{busCode\}$.

We can replace the set on the left by a set that contains it.

The final rule, 5, represents a rather more subtle way of deriving functional dependencies. This is referred to as **derivation by transitivity**, and is illustrated by the following example.

EXAMPLE 2.2

In the discussion of Activity 2.1 we obtained the following functional dependencies for the scheme **STUDENT_TUTOR**.

- FD1: $\{tutorID\} \rightsquigarrow \{tutorEmail\}$.
- FD2: $\{studentID\} \rightsquigarrow \{studentEmail\}$.
- FD3: $\{studentID, courseCode\} \rightsquigarrow \{tutorID\}$.

FD3 states that if we know the values of *studentID* and *courseCode* then we will know the value of *tutorID*. But FD1 then states that if we know the value of *tutorID* we will know the value of *tutorEmail*. Combining these, if we know the values of *studentID* and *courseCode* then we will know the value of *tutorEmail*. Thus we have a functional dependency

$$\{studentID, courseCode\} \rightsquigarrow \{tutorEmail\}.$$

Each of the rules in Result 2.1 comes from a fairly simple idea. But if the ideas are combined, it can become a little difficult to be sure whether some argument about a new functional dependency is correct. In such a case, you can use Result 2.1 to check that each step in your argument is correct.

EXAMPLE 2.3

In Example 2.1 we considered the scheme **BUS**, and showed that

- FD2: $\{driverID, day\} \rightsquigarrow \{busCode\}$,
- FD3: $\{busCode, day\} \rightsquigarrow \{routeCode\}$.

So $\{driverID, day\}$ determines both *day* (trivially) and *busCode* (by FD2), and so must determine *routeCode* (by FD3).

We can be sure that this is a valid argument by setting it out as below.

Since $\{day\} \rightsquigarrow \{day\}$ (Result 2.1(1)), we can use Result 2.1(4) to obtain

$$FD4: \{driverID, day\} \rightsquigarrow \{day\}.$$

We can then use Result 2.1(2) on FD2 and FD4 to obtain

$$FD5: \{driverID, day\} \rightsquigarrow \{busCode, day\}.$$

We can then apply Result 2.1(5) to FD5 and FD3, to obtain

$$\{driverID, day\} \rightsquigarrow \{routeCode\}.$$

This sort of argument is acceptable in your own work.

Use the more formal approach if you want to confirm for yourself that an argument is valid.

Activity 2.2

For the scheme **BARCODEITEM** on page 16, determine what functional dependencies follow from the give rules. (The attribute *cost* gives the price in pence to be charged for the product. The attribute *product* is a sufficiently full description to determine *cost*, such as “Own brand Wheatflakes, Large”. The attribute *barcode* contains enough information to determine *product*, and may contain additional information, such as the “best before” date.)

BARCODEITEM			Relation scheme
Attribute	Domain	Type	
<i>barcode</i>	<i>Barcode</i>	Barcode	
<i>product</i>	<i>ProductName</i>	mString	
<i>cost</i>	<i>Price</i>	Int	
<i>Rules</i>			
R1 The <i>product</i> contains enough information to determine the price.			
R2 The <i>product</i> can be determined from the <i>barcode</i> , and the <i>barcode</i> contains additional information.			

Discussion If we know the product then we know the price (R1), so
FD1: $\{product\} \rightsquigarrow \{cost\}$.
From R2, if we know the barcode then we know the product, so
FD2: $\{barcode\} \rightsquigarrow \{product\}$.
R2 also indicates that the product does not determine the barcode; that is,
 $\{product\} \not\rightsquigarrow \{barcode\}$.
You may have also observed that if we know the barcode then we know the price; that is, $\{barcode\} \rightsquigarrow \{cost\}$. This dependency can be deduced from FD1 and FD2 by transitivity.

We can separate the functional dependencies of a relation scheme into two groups. There are those that follow directly from the rules of the scheme. We will call these the **explicit functional dependencies**. We will call all the remaining functional dependencies, which can be obtained using Result 2.1, **implicit functional dependencies**. These implicit functional dependencies include **trivial** dependencies such as $\{x\} \rightsquigarrow \{x\}$ (which would hold in any scheme with an attribute x), and any dependencies that can be deduced from the explicit and trivial dependencies using rules like transitivity.

The explicit dependencies are a result of the meaning of the attributes. As part of the description of a scheme, we have to give enough information to allow the explicit dependencies to be determined. This can be done from rules, as we have done so far. In some future examples, we may simply give the explicit dependencies directly, as part of our description of a scheme. However we describe a scheme, you may assume that the functional dependencies consist only of the explicit dependencies (given directly or deduced from the given rules), and the implicit dependencies derivable from them.

The implicit dependencies that are most easily missed are those that come from transitivity. It can be useful to have a technique for generating all the functional dependencies in a relation scheme, both explicit and implicit. One way of doing this is to use a table. This technique is illustrated in the next example. The rows and columns of the table show all the possible non-empty subsets of the attributes. A $\checkmark$ is inserted at a location in the table when the set of attributes labelling the column is determined by the set of attributes labelling the row.

It would be sufficient just to use the columns labelled with single attributes. However, when we use derivation by transitivity it is much easier to see where ticks need adding if we have all the subsets as labels.

EXAMPLE 2.4

We will draw up a table showing all the functional dependencies for the scheme **BARCODEITEM** considered in Activity 2.2. There, we obtained the following dependencies from the rules of the scheme.

FD1: $\{product\} \rightsquigarrow \{cost\}$.
FD2: $\{barcode\} \rightsquigarrow \{product\}$.

This scheme has three attributes, and a set with 3 elements has $2^3 - 1$ non-empty subsets. So the table will have 7 rows and columns. To save space we use single letters for the attributes and show subsets containing more than one attribute by writing the appropriate letters together. We write B for *barcode*, P for *product*, C for *cost*, BP for $\{barcode, product\}$, and so on.

We insert a tick into a cell of the table if the row label determines the column label. We will start by filling in the ticks that will hold for any relation scheme. Thus we can add ticks to all the main diagonal cells since $A \rightsquigarrow A$ for any set of attributes A . We can also add a tick in a row labelled A to the column labelled X whenever $X \subseteq A$. For example, we add ticks in the row labelled BP to the columns labelled B, P. These steps rely on

The trivial dependencies.

For example, $\{B, P\} \rightsquigarrow \{P\}$.

	B	P	C	BP	BC	PC	BPC
B	✓						
P		✓					
C			✓				
BP	✓	✓		✓			
BC	✓		✓		✓		
PC		✓	✓			✓	
BPC	✓	✓	✓	✓	✓	✓	✓

We now add in any ticks that come from the rules of the relation scheme.

The explicit dependencies.

	B	P	C	BP	BC	PC	BPC
B	✓	✓					
P		✓	✓				
C			✓				
BP	✓	✓		✓			
BC	✓		✓		✓		
PC		✓	✓			✓	
BPC	✓	✓	✓	✓	✓	✓	✓

At each stage, the new ticks are highlighted for clarity.

Next we use Result 2.1 (2). In each row we put a tick in a column that is labelled by a set of attributes if all of the attributes in the set already have ticks in that row.

We now look for implicit dependencies.

	B	P	C	BP	BC	PC	BPC
B	✓	✓		✓			
P		✓	✓			✓	
C			✓				
BP	✓	✓		✓			
BC	✓		✓		✓		
PC		✓	✓			✓	
BPC	✓	✓	✓	✓	✓	✓	✓

Next we use Result 2.1 (5). In our example the row labelled B has a tick in the column labelled P. Also, the row P has ticks in the columns C and PC. So in the row labelled B, we must add ticks to the columns labelled C and PC. (This is because $B \rightsquigarrow P$ and $P \rightsquigarrow \{P, C\}$, so that $B \rightsquigarrow \{P, C\}$ by transitivity.) We need to check throughout the table. In general, the approach is as follows. Move down the table looking at each row in turn. Suppose you find a row labelled X with a tick in the column labelled Y , and in the row labelled Y there is a tick in the column labelled Z . Then you must add a tick in row X to the column labelled Z . We find that various additions are needed here. Row BP has a tick in column P, and looking at row P, we see that ticks should be added to row BP in columns C and PC. Row BC has a tick in column B and row B has ticks in columns P and BP, so row BC needs ticks added in columns P and BP.

Look for dependencies that can be deduced by transitivity.

These could have been deduced by using Result 2.1(4), but we can also find them using transitivity.

	B	P	C	BP	BC	PC	BPC
B	✓	✓	✓	✓		✓	
P		✓	✓			✓	
C			✓				
BP	✓	✓	✓	✓		✓	
BC	✓	✓	✓	✓	✓		
PC		✓	✓			✓	
BPC	✓	✓	✓	✓	✓	✓	✓

Finally we use Result 2.1 (2 and 3) again and ensure that there is a tick in a column for a set of attributes precisely when there is a tick in all of the columns for the attributes of that set. This is necessary after adding the dependencies due to transitivity.

	B	P	C	BP	BC	PC	BPC
B	✓	✓	✓	✓	✓	✓	✓
P		✓	✓			✓	
C			✓				
BP	✓	✓	✓	✓	✓	✓	✓
BC	✓	✓	✓	✓	✓	✓	✓
PC		✓	✓			✓	
BPC	✓	✓	✓	✓	✓	✓	✓

To demonstrate that the table is fully complete we will verify that no functional dependency exists where there is no tick. Consider the sample tuples in the table below.

	barcode	product	cost
t1	33050	2.5 kg organic potatoes	299
t2	53151	400 g mature cheddar	299
t3	77502	Own brand Wheatflakes, Large	99
t4	77505	Own brand Wheatflakes, Large	99

First note that these tuples satisfy the explicit dependencies FD1 and FD2. They satisfy FD1 since where the two products are the same (in t3 and t4) so are the costs. Since all the barcodes are different, FD2 is bound to be satisfied.

Tuples *t1* and *t2* have the value of *cost* the same while the values of *barcode* and of *product* are different. So they show that there should be only one tick in the row labelled C. Tuples *t3* and *t4* have *product* (and *cost*) the same and different values of *barcode*. So they show that there should be no more ticks in the rows labelled P and PC.

The final table of dependencies shows more information than is necessary. In the final table we only need the columns headed by single attributes. In principle, we do not need some of the rows either (those that give no new information), but it is not easy to see which rows can safely be omitted. In this example, the table on the right encapsulates the key information. It gives the explicit dependencies, plus one crucial additional dependency that comes from transitivity.

So $\{cost\} \not\rightarrow \{product\}$
and $\{cost\} \not\rightarrow \{barcode\}$.

	B	P	C
B	✓	✓	✓
P		✓	✓

The procedure we followed in the above example is not one that can easily be used by hand for a scheme with more than three attributes. But it does indicate how we might automate the process of obtaining all functional dependencies from the explicit dependencies (using Result 2.1). We can summarise the procedure we used as below.

- Fill in the trivial dependencies.
- Add in the explicit functional dependencies.
- Tick any column for a set of attributes where the column for each individual attribute is already ticked.
- Add any functional dependencies that come from transitivity.
- Ensure that there is a tick in a column for a set of attributes precisely when there is a tick in all of the columns for the attributes of that set.

Result 2.1 (1 and 3)
These come from the rules.

Result 2.1 (2)
Result 2.1 (5)

Result 2.1 (2 and 3)

If one had a large set of attributes, it might be necessary to go through this process more than once. In your own work, you may find it useful to summarise the crucial dependencies in as short a list as possible. This should include the explicit dependencies, and additional dependencies that can be deduced by transitivity. For example, for **BARCODEDITEM**, we might summarise the crucial dependencies as below.

FD1: $\{product\} \rightsquigarrow \{cost\}$.
FD2A: $\{barcode\} \rightsquigarrow \{product, cost\}$.

EXAMPLE 2.5

In Example 2.2 we gave the following explicit functional dependencies for the scheme **STUDENT_TUTOR**.

- FD1: $\{tutorID\} \rightsquigarrow \{tutorEmail\}$.
- FD2: $\{studentID\} \rightsquigarrow \{studentEmail\}$.
- FD3: $\{studentID, courseCode\} \rightsquigarrow \{tutorID\}$.

We used a “commonsense” approach to deduce that we must have the additional dependency

$\{studentID, courseCode\} \rightsquigarrow \{tutorEmail\}$.

Incidentally, the sort of argument used there to deduce this dependency is quite acceptable in your own work. It can be justified formally by reference to Result 2.1(5). But we will not usually ask for such a formal justification. In practice, use of Result 2.1 provides a “safety net”, which you can use to verify your own intuition about implicit dependencies.

For the scheme **STUDENT_TUTOR**, the following might provide a useful list of “crucial” dependencies.

- FD1: {*tutorID*} $\rightsquigarrow$ {*tutorEmail*}.
- FD2: {*studentID*} $\rightsquigarrow$ {*studentEmail*}.
- FD3B: {*studentID*, *courseCode*} $\rightsquigarrow$ {*tutorID*, *tutorEmail*, *studentEmail*}.

If you are unsure about how the implicit dependencies here were found, see Example 2.6 on page 21.

Activity 2.3

Explain why the tuples in the following table could occur together in a relation over the scheme **STUDENT_TUTOR**. Use appropriate tuples from the table to explain why {*studentID*} $\nrightarrow$ {*courseCode*}, {*courseCode*} $\nrightarrow$ {*tutorID*}, and {*courseCode*, *tutorID*} $\nrightarrow$ {*studentID*}.

Recall that the notation $\nrightarrow$ indicates that a functional dependency does not hold.

	<i>courseCode</i>	<i>studentID</i>	<i>tutorID</i>	<i>studentEmail</i>	<i>tutorEmail</i>
t1	M263	A1234	0123	qjh3@st.ac.uk	tmj2@tu.ac.uk
t2	M263	A1245	0124	qjh3@st.ac.uk	ckj2@tu.ac.uk
t3	M263	A1257	0123	abc3@st.ac.uk	tmj2@tu.ac.uk
t4	M264	A1245	0137	qjh3@st.ac.uk	adv4@tu.ac.uk

Discussion We need to check that the table satisfies the explicit dependencies FD1, FD2 and FD3. FD1 is satisfied since when the tutor ID codes are the same (in t1 and t3), so are the tutor email addresses. FD2 is satisfied since when the student ID codes are the same (in t2 and t4), so are the student email addresses. No two tuples have the same values for both *studentID* and *courseCode*, so FD3 is automatically satisfied.

Tuples t2 and t4 have the same values of *studentID* but different values of *courseCode*, and so show that

{*studentID*} $\nrightarrow$ {*courseCode*}.

Tuples t1 and t2 have the same values of *courseCode* but different values of *tutorID*, and so show that

{*courseCode*} $\nrightarrow$ {*tutorID*}.

Tuples t1 and t3 have the same values of both *courseCode* and *tutorID* but different values of *studentID*, and so show that

{*courseCode*, *tutorID*} $\nrightarrow$ {*studentID*}.

Incidentally, you may have noticed that tuples t1 and t2 have the same values of *studentEmail* and different values of *studentID* (and so show that {*studentEmail*} $\nrightarrow$ {*studentID*}). The rules allow this — two students may share an email address.

The implicit dependencies that are most easily overlooked are those that come from transitivity. If you are not sure whether you have found these, then Strategy 2.1 below can be used. An example illustrating its use is given after the strategy.

Strategy 2.1**Finding implicit dependencies that come from transitivity**

1. Augment each explicit functional dependency by adding all the attributes from the left-hand side to the right-hand side.
2. For each augmented dependency, check whether the left-hand side is a subset of the right-hand side of any other dependency.
3. If it is, then you will have dependencies $X \rightsquigarrow Y$ and $W \rightsquigarrow Z$, where $W \subseteq Y$. Add $X \rightsquigarrow Z$ to your list. Then augment this new dependency, as in step 1.
4. Amalgamate any dependencies that have the same left-hand side using Result 2.1 (2).
5. Repeat steps 2–4 until all the dependencies have been checked.
6. Finally, reduce all the functional dependencies by removing from the right-hand side any attributes on the left-hand side.

EXAMPLE 2.6

The explicit dependencies of **STUDENT_TUTOR** are repeated below.

FD1: $\{studentID\} \rightsquigarrow \{studentEmail\}$.

FD2: $\{tutorID\} \rightsquigarrow \{tutorEmail\}$.

FD3: $\{studentID, courseCode\} \rightsquigarrow \{tutorID\}$.

We use Strategy 2.1 to obtain further functional dependencies. The first step is to augment the explicit dependencies. This gives:

FD1A: $\{studentID\} \rightsquigarrow \{studentEmail, studentID\}$.

FD2A: $\{tutorID\} \rightsquigarrow \{tutorEmail, tutorID\}$.

FD3A: $\{studentID, courseCode\} \rightsquigarrow \{tutorID, studentID, courseCode\}$.

We now check whether the left-hand side of a dependency is a subset of a right-hand side of any other dependency. The left-hand side of FD1A is a subset of the right-hand side of FD3A. This gives the dependency

FD4: $\{studentID, courseCode\} \rightsquigarrow \{studentEmail, studentID\}$.

This augments to

FD4A: $\{studentID, courseCode\} \rightsquigarrow \{studentID, courseCode, studentEmail\}$.

The left-hand side of FD2A is a subset of the right-hand side of FD3A. So

FD5: $\{studentID, courseCode\} \rightsquigarrow \{tutorEmail, tutorID\}$.

This augments to

FD5A: $\{studentID, courseCode\} \rightsquigarrow \{tutorID, studentID, courseCode, tutorEmail\}$.

The next step amalgamates FD3A, FD4A and FD5A.

FD3B: $\{studentID, courseCode\} \rightsquigarrow \{studentID, courseCode, tutorID, tutorEmail, studentEmail\}$.

The procedure gives no further dependencies. The list of reduced dependencies is as below.

FD1: $\{studentID\} \rightsquigarrow \{studentEmail\}$.

FD2: $\{tutorID\} \rightsquigarrow \{tutorEmail\}$.

FD3C: $\{studentID, courseCode\} \rightsquigarrow \{tutorID, tutorEmail, studentEmail\}$.

2.3 Keys

In Example 2.4 we saw that $\{barcode\}$ determines the other attributes, *product* and *cost*, of the scheme **BARCODEITEM**. If two tuples had the same value of *barcode*, then the values of all the attributes would also be equal, and so the tuples would be identical. So the rules of this scheme mean that a relation cannot include two tuples with the same value of *barcode*.

We would not include identical tuples in a table.

In the relation scheme **BUS** (Example 2.1), no single attribute determines all the others. In that case, $\{day, driverID\}$ determines all the other attributes. In general, we say that any attribute or set of attributes that determine all the other attributes of a scheme has the **determining property**.

See Example 2.3.

Now in **BUS**, it is also true that if we know the values of all three of *day*, *driverID* and *busCode* then we know the values of all the attributes. However, in that case we have more information than is necessary. Note also that the attribute *day* on its own is not sufficient to determine the other attributes, nor is *driverID* sufficient. We say that the set of attributes $\{day, driverID\}$ has the **minimal property**.

The set $\{day, driverID, busCode\}$ does not have the minimal property.

Important sets of attributes are those that determine all the rest, and do not have more information than necessary. Such sets have both the determining and the minimal properties. A set of attributes with both these properties is called a **key** of a relation scheme.

In a relation scheme, a subset K of the set of attributes Att is a **key** for the relation scheme if both of these properties hold.

- For every attribute x in Att , $K \rightsquigarrow \{x\}$. **(Determining)**
- There is no proper subset $Y \subset K$ with the property that $Y \rightsquigarrow \{x\}$ for all x in Att . **(Minimal)**

Recall that a proper subset of K is not equal to K .

Activity 2.4

Explain why $\{day, busCode\}$ is a key of the relation scheme **BUS**.

You can use results obtained in Example 2.1.

Discussion We saw in Example 2.1 that $\{day, busCode\}$ determines both *driverID* and *routeCode*. Consequently, this set of attributes has the determining property. We can show that it has the minimal property by showing that neither $\{day\}$ nor $\{busCode\}$ has the determining property. To do this, refer to the sample relation given in the table on page 13. Tuples t1 and t2 show that *day* does not determine any of the other attributes. Tuples t3 and t7 show that *busCode* does not determine *day*, and so does not have the determining property.

Since $\{day, busCode\}$ has both the determining property and the minimal property, it is a key of **BUS**.

We noted earlier that $\{day, driverID\}$ has both the determining and minimal properties, and so is also a key of **BUS**. So **BUS** has two keys.

Where a relation scheme has more than one key, it is usual to declare one of these as the **primary key** of the relation scheme. There is no standard procedure for deciding which key should be the primary key. It depends upon the context within which the database is being developed.

The keys of the scheme may then be referred to as **candidate keys**.

Sometimes, we need to find all the keys of a scheme. To be sure that we have all the keys of **BUS** takes more effort. Note first that the sample tuples in the table on page 13 show that no single attribute has the determining property. Also, tuples t3 and t7 have all three attributes *busCode*, *driverID* and *routeCode* the same, but the attribute *day* different. This shows that $\{busCode, driverID, routeCode\} \not\rightarrow \{day\}$. As all the other attributes together do not determine *day*, we see that any key must contain *day*. Finally, tuples t6 and t7 show that $\{day, routeCode\}$ does not have the determining property. This eliminates all remaining possibilities.

Activity 2.5

A relation scheme **HORSE**, and a table showing a partial relation over the scheme, are given below.

HORSE			Relation scheme
Attribute	Domain	Type	
<i>name</i>	<i>HorseName</i>	mString	
<i>dam</i>	<i>HorseName</i>	mString	
<i>sire</i>	<i>HorseName</i>	mString	
<i>birthYear</i>	<i>Year</i>	Int	
<i>gender</i>	{‘M’, ‘F’}	Char	
<i>Rules</i>			
R1: No two horses have the same name.			
R2: No female horse has more than one foal per year.			

HORSE might be used by a stud farm (a farm that breeds and raises horses).

The female parent of a horse is called its dam and the male parent is called its sire. A young horse is called a foal.

	<i>name</i>	<i>dam</i>	<i>sire</i>	<i>birthYear</i>	<i>gender</i>
t1	FlashHarry	FlashLady	Gordon	1995	M
t2	PrettyOne	FlashLady	FastBoy	1996	F
t3	SteadyGirl	WhiteHoof	FastBoy	1996	F
t4	Frisky	FlashLady	Gordon	1997	M
	⋮	⋮	⋮	⋮	⋮

- (a) Find all the explicit functional dependencies of **HORSE**.
- (b) Show that $\{dam, birthYear\}$ has the determining property.
- (c) Find two keys of the scheme **HORSE**.

Discussion

- (a) The first rule tells us that a horse’s name is a unique identifier for the horse. Then common sense tells us that a horse has a unique dam, sire, gender and birth year. Hence R1 gives the dependency

FD1: $\{name\} \rightsquigarrow \{dam, sire, gender, birthYear\}$.

The second rule tells us that if we know the dam and the year then we know the horse, so:

FD2: $\{dam, birthYear\} \rightsquigarrow \{name\}$.

- (b) Combining FD2 and FD1 using transitivity shows that:

$\{dam, birthYear\} \rightsquigarrow \{dam, sire, gender, birthYear\}$.

Together with FD2, this shows that $\{dam, birthYear\}$ has the determining property.

Result 2.1 (5).

- (c) FD1 shows that $\{name\}$ determines all the attributes, so $\{name\}$ has the determining property required by a key. Since it is a single attribute it also has the minimal property. Thus $\{name\}$ is a key for the scheme.
- We showed in (b) that $\{dam, birthYear\}$ has the determining property. Now tuples t1 and t2 show that $\{dam\}$ does not have the determining property, while t2 and t3 shows that $\{birthYear\}$ does not have the determining property. So $\{dam, birthYear\}$ also has the minimal property, and so is a second key.

In **HORSE**, the tuples t2 and t3 show that $\{sire, birthYear, gender\}$ does not have the determining property and t1 and t4 show that $\{dam, sire, gender\}$ does not have the determining property. Any other set of attributes will either be a subset of one of these two sets (and so will not have the determining property), or will contain either $\{name\}$ or $\{dam, birthYear\}$ as a proper subset (and so will not be minimal). So the only keys of **HORSE** are $\{name\}$ and $\{dam, birthYear\}$.

Objectives for Section 2

After studying this section you should be able to do the following.

- ☐ Understand the terminology: functional dependency; explicit, implicit and trivial dependencies; dependency derived by transitivity; determining property; minimal property; key.
- ☐ Use the rules of a scheme to identify its explicit functional dependencies.
- ☐ Determine implicit functional dependencies that hold (using Result 2.1).
- ☐ Choose or create tuples to show that a specified functional dependency does not hold.
- ☐ Given the explicit dependencies, find the key(s) of a scheme.

Exercises on Section 2

Exercise 2.1

Give a key for the scheme **STUDENT_TUTOR**. Explain why it is a key. (There is no need to go over the derivation of the implicit dependencies of this scheme that are covered in the text.)

You can refer to Example 2.5 and Activity 2.3.

Exercise 2.2

Identify the explicit functional dependencies for the relation scheme **STUDENT**. Describe all the dependencies of this scheme. Construct example tuples that show that there are no other dependencies. Determine all the keys for this relation scheme.

See page 7.
Assume that different students may have the same *name* and the same *email*.

Exercise 2.3

For the relation scheme **CENTRE_TIMETABLE**, identify the explicit functional dependencies. Determine all the keys for this scheme.

See page 9.

3 Linking schemes by foreign keys

All the examples so far have consisted of one relation scheme, with relations in one table. For many databases, this is not a good design. Consider, for example, a relation scheme about students and courses, in a situation where a student may take several courses. Such a scheme would record details of the student, such as name and address, for each course the student takes. If a student changes address, their details would then need updating for every relevant entry in the table. This is both inefficient and a possible source of error. In order to avoid such problems, E.F.Codd introduced the idea of a **relational database**. A relational database consists of a collection of relation schemes linked together. In this section we will consider briefly some of the main ideas involved in such structures. As a first example, consider a relational database that includes the three relation schemes below.

Codd, E. F. “A relational model of data for large shared data banks” *Communications of the Association for Computing Machinery* Vol. 13, No. 6, June 1970, pp. 377–387.

STUDENT			Relation scheme
Attribute	Domain	Type	
<i>studentID</i>	<i>StudentIDCode</i>	mString	
<i>name</i>	<i>StudentName</i>	mString	
<i>postcode</i>	<i>Postcode</i>	mString	
<i>Rules</i>			
R1 A student cannot have more than one name or postcode.			

COURSE			Relation scheme
Attribute	Domain	Type	
<i>courseCode</i>	<i>CourseCode</i>	mString	
<i>title</i>	<i>CourseTitle</i>	mString	
<i>credit</i>	{10, 15, 20, 30, 60}	Int	
<i>Rules</i>			
R1 A course cannot have more than one title or credit rating.			

RESULT			Relation scheme
Attribute	Domain	Type	
<i>studentID</i>	<i>StudentIDCode</i>	mString	
<i>courseCode</i>	<i>CourseCode</i>	mString	
<i>grade</i>	{1, 2, 3, 4, 5, Null}	Grade	
<i>Rules</i>			
R1 No student can have two results for a course.			

In these, *studentID* and *courseCode* are unique identifiers. We assume that a student may have results for several courses. A key for the first scheme is $\{studentID\}$, a key for the second scheme is $\{courseCode\}$, and a key for the third scheme is $\{studentID, courseCode\}$. These are the only keys of each scheme, and so must be the primary key in each case.

We wish to investigate how these schemes are related. A student may have results for several courses. Thus for each tuple over **STUDENT** there can be several corresponding tuples over **RESULT**. This situation is illustrated by the partial table below. This shows relations over **STUDENT** and **RESULT**, and how tuples over the two schemes are linked. Notice that the attribute *studentID* appears twice — once in each scheme.

Student Table			Result Table		
<i>studentID</i>	<i>name</i>	<i>postcode</i>	<i>studentID</i>	<i>courseCode</i>	<i>grade</i>
M1234	Hamid	LE41 9ZY	M1234	C123	Null
			M1234	C224	1
			M1234	C421	2
M3456	Jones	LE41 9ZY	M3456	C123	Null
			M3456	C224	4
M3577	Jones	NL99 2QA	M3577	C224	Null
⋮	⋮	⋮	⋮	⋮	⋮

We say that there is a **one to many association** between the schemes **STUDENT** and **RESULT**. Each student may have results for “many” courses (where “many” is used to mean “zero, one, or more than one”). But each result belongs to just one student. So one tuple in the Student table is linked to many tuples in the Results table.

Now $\{studentID\}$ is a key of the scheme **STUDENT**. So the attributes in the scheme **RESULT** include a key of a different relation scheme, namely **STUDENT**. We say that $\{studentID\}$ is a **foreign key** in the relation scheme **RESULT**. This is what links the two relation schemes together. Each tuple over **RESULT** is linked to a unique tuple over **STUDENT** — that with the appropriate value of the foreign key *studentID*.

Many texts use the word relation where we have association. We have used association to avoid confusion between different uses of the word relation.

As illustrated in the tables above.

A **foreign key** is a set of attributes in one relation scheme **R** that is the primary key of another relation scheme **S**. For there to be a foreign key, **S** and **R** must have a one to many association. We say that the foreign key in the scheme **R** **references** the scheme **S**.

In the example above, the attribute *studentID* is part of the primary key of **RESULT**. However this is not necessary. A foreign key may — or may not — be contained in the primary key of the linked relation scheme.

Since each course can be studied by many students, it is also the case that there is a one to many association between **COURSE** and **RESULT**.

Activity 3.1

The table below shows part of a relation over the scheme **COURSE**. Use the tuples given below and those given earlier for **RESULT** to illustrate the one to many association between the schemes **COURSE** and **RESULT**. What is the foreign key?

Course Table		
<i>courseCode</i>	<i>title</i>	<i>credits</i>
C123	Databases	30
C224	HCI	15
C421	AI	60
⋮	⋮	⋮

Discussion We combine the information given so that the tuples from the Result Table are listed beside the tuples with the same course code.

Course Table			Result Table		
<i>courseCode</i>	<i>title</i>	<i>credits</i>	<i>studentID</i>	<i>courseCode</i>	<i>grade</i>
C123	Databases	30	M1234	C123	Null
			M3456	C123	Null
			⋮	⋮	⋮
C224	HCI	15	M1234	C224	1
			M3456	C224	4
			M3577	C224	Null
C421	AI	60	⋮	⋮	⋮
			M1234	C421	2
			⋮	⋮	⋮

There are likely to be other students for each course, as well as those given. Hence the dots — to indicate that the Result Table is incomplete.

The primary key of the relation scheme **COURSE** is {*courseCode*}, and this attribute is in the scheme **RESULT**. Hence the foreign key in the scheme **RESULT** is {*courseCode*}. Notice that **RESULT** contains two different foreign keys, for the two different linked schemes.

A relation scheme **OBSERVATION** is given below. This records sightings of birds, with the place and date, and any related notes. The rule for this scheme indicates that the set of attributes {*commonName*, *site*, *date*} has the determining property. A sample table is also given below.

OBSERVATION			Relation scheme
Attribute	Domain	Type	
<i>commonName</i>	<i>BirdName</i>	mString	
<i>site</i>	<i>Place</i>	mString	
<i>date</i>	<i>Date</i>	Date	
<i>notes</i>	<i>SeqOfChar</i>	mString	
Rules			
R1 Two sightings cannot have all of bird name, site and date the same.			

Observation Table

	<i>commonName</i>	<i>site</i>	<i>date</i>	<i>notes</i>
t1	Dipper	Lathkill Dale	14/02/03	lots of them
t2	Kingfisher	Tim's garden	12/08/01	what luck!
t3	Dipper	Bradford Dale	14/02/03	flew past
t4	Blackbird	Our garden	30/03/03	nest building
t5	Kingfisher	Bradford Dale	14/02/03	flash of orange
t6	Blackbird	Our garden	1/04/03	eyeing the cherry tree
t7	Mockingbird	Bradford Dale	1/04/03	April fool!
t8	Kingfisher	Tim's garden	21/06/02	carrying a fish
t9	Kingfisher	Tim's garden	21/07/02	youngsters!

The tuples above show that no proper subset of $\{commonName, site, date\}$ has the determining property. (t1 and t3 rule out $\{commonName, date\}$, t4 and t6 rule out $\{commonName, site\}$, t3 and t5 rule out $\{site, date\}$.) Thus $\{commonName, site, date\}$ is a key for **OBSERVATION**.

The scheme **OBSERVATION** shares an attribute, *commonName*, with the scheme **BIRD** given in Exercise 1.1. **BIRD** has a one to many association with **OBSERVATION**, with $\{commonName\}$ as the foreign key.

Suppose we have another relation scheme **SITE** that records good sites for bird watching, and times of year when they are good. A sample table for this scheme is also given below.

SITE		Relation scheme
Attribute	Domain	Type
<i>commonName</i>	<i>BirdName</i>	mString
<i>site</i>	<i>Place</i>	mString
<i>season</i>	{Spring, Summer, Autumn, Winter}	mString

Site Table

	<i>commonName</i>	<i>site</i>	<i>season</i>
s1	Dipper	Lathkill Dale	Spring
s2	Dipper	Bradford Dale	Spring
s3	Kingfisher	Tim's garden	Summer
s4	Kingfisher	Tim's garden	Winter
s5	Blackbird	Our garden	Spring
s6	Dipper	Lathkill Dale	Summer
s7	Puffin	Bempton Cliffs	Summer

There are no rules about tuples that may occur together in a relation over this scheme. As there are no rules there are no functional dependencies. The only key of **SITE** is $\{commonName, site, season\}$. The schemes **BIRD** and **SITE** again share one attribute, *commonName*, and also have a one to many association, with *commonName* in **SITE** the foreign key.

Except the trivial dependencies.

However the relation schemes **OBSERVATION** and **SITE** do not have a one to many association. For example, t1 in the Observation Table corresponds to both s1 and s6 in the Site Table, while s5 in the Site Table corresponds to t4 and t6 in the Observation Table. Indeed, t2, t8 and t9 in the Observation Table all match both s3 and s4 in the Site Table.

We regard tuples in the two schemes as “corresponding” if they have the same values for the attributes that the schemes have in common: here *commonName* and *site*.

These relation schemes are said to have a **many to many** association. It is not possible to link two such schemes by using a foreign key. (They do have an indirect link, through **BIRD**.)

Now suppose that a library has a database containing the relation schemes **BOOK** and **ORDER** given below.

BOOK			Relation scheme
Attribute	Domain	Type	
<i>isbn</i>	<i>ISBN</i>	ISBNCode	
<i>author</i>	<i>AuthorName</i>	mString	
<i>title</i>	<i>Title</i>	mString	
<i>Functional dependencies</i>			
FD1: { <i>isbn</i> } $\rightsquigarrow$ { <i>author</i> , <i>title</i> }			

ORDER			Relation scheme
Attribute	Domain	Type	
<i>isbn</i>	<i>ISBN</i>	ISBNCode	
<i>onOrder</i>	<i>Bool</i>	Bool	
<i>Functional dependencies</i>			
FD1: { <i>isbn</i> } $\rightsquigarrow$ { <i>onOrder</i> }			

The *isbn* identifies a publication completely. Two published books might share both author and title, but they will have different values of *isbn*. In **ORDER**, *onOrder* is **true** if that book is on order.

Tuples in relations over these two schemes will correspond if they have the same value of the common attribute, *isbn*. Now in each scheme, {*isbn*} is a key (and is the only key in each case). So a particular value of *isbn* will never occur in more than one tuple in a relation over either scheme. We say that these two schemes have a **one to one** association. (A tuple in a relation over one scheme will correspond with at most one tuple over the other scheme.) In this case, the choice of foreign key to link the schemes is rather confusing. We can designate *isbn* as a foreign key of **ORDER** in **BOOK** — or a foreign key of **BOOK** in **ORDER**. In this situation, *isbn* is often called an **alternate key**. The choice is essentially arbitrary.

A relational database will usually consist of a collection of relation schemes. Each of these schemes is likely to have one to many or one to one associations with one or more other schemes in the database. These associations will be indicated by declaring foreign keys.

There may be several reasons for this division of the database into a collection of linked schemes (rather than a single scheme). With a large scheme, the tables showing relations will become extremely unwieldy. A key issue was mentioned in the introduction to this unit. This is to avoid **data duplication**. With a poorly designed scheme, one may have, for example, data such as a student’s email address kept in a number of different tuples. If a change of email address is made, then all these tuples would need to be located and changed. A collection of schemes, with appropriate links, can avoid such problems. In the next section, we look at how we can anticipate such problems, and subdivide a scheme so as to avoid them.

Sometimes the underlying data suggests which one to choose, but this is an issue of database design that we will not cover in this course.

To fully describe the mechanisms of linking relation schemes it is necessary to declare further rules, which we shall not discuss.

Objectives for Section 3

After studying this section you should be able to do the following.

- ☐ Understand that a relational database consists of interconnected tables.
- ☐ Identify a foreign key.
- ☐ Classify associations between schemes (as one to many, etc).

Exercises on Section 3

Exercise 3.1

Suppose that the library mentioned in the text above gives each book that it owns a library identifier *bookID* (it may own several copies of the same book). As well as **BOOK** and **ORDER**, its database has two further schemes.

STOCK, with attributes *isbn* and *bookID*.

LOAN, with attributes *bookID* and *inLibrary*.

The attribute *inLibrary* is **true** if the book with library identifier *bookID* is currently on the library shelves available for loan. **STOCK** records the *isbn* of each book that the library owns.

Give a key for each of **STOCK** and **LOAN**. What sort of association is there between these two schemes, and between **STOCK** and **BOOK**? Identify a foreign key in each case.

Exercise 3.2

The two relation schemes given below form part of a supermarket database. Identify a key for each relation scheme. Describe the association between the schemes and identify a foreign key.

TILL			Relation scheme
Attribute	Domain	Type	
<i>tillID</i>	<i>TillCode</i>	mString	
<i>tillType</i>	{Wide, Standard}	mString	
<i>tillMake</i>	<i>Manufacturers</i>	mString	
<i>Functional dependencies</i>			
FD1: { <i>tillID</i> } $\rightsquigarrow$ { <i>tillType</i> , <i>tillMake</i> }			

TILL gives details relating to an individual till.

TRANSACTION			Relation scheme
Attribute	Domain	Type	
<i>transID</i>	<i>TransCode</i>	mString	
<i>tillID</i>	<i>TillCode</i>	mString	
<i>total</i>	<i>Price</i>	Int	
<i>cashierID</i>	<i>Cashier</i>	mString	
<i>Functional dependencies</i>			
FD1: { <i>transID</i> } $\rightsquigarrow$ { <i>tillID</i> , <i>total</i> , <i>cashierID</i> }			

TRANSACTION gives details of transactions.

4 Normal forms

In this section we shall consider some features of a relation scheme that make the scheme easier to maintain. It would clearly not be sensible to record all the information for a university database in one relation scheme. Apart from the unwieldy number of attributes this would give, there would also be considerable data duplication. For example, a sample table over the scheme **STUDENT_TUTOR**, is given below.

See page 11.

<i>courseCode</i>	<i>studentID</i>	<i>tutorID</i>	<i>studentEmail</i>	<i>tutorEmail</i>
M263	A1234	0123	qjh3@st.ac.uk	tmj2@tu.ac.uk
M263	A1245	0124	qjh3@st.ac.uk	ckj2@tu.ac.uk
M263	A1257	0123	abc3@st.ac.uk	tmj2@tu.ac.uk
M264	A1257	0123	abc3@st.ac.uk	tmj2@tu.ac.uk
M264	A2766	0123	dtv1@st.ac.uk	tmj2@tu.ac.uk

A relation over this scheme will duplicate the email address of each student and tutor many times. (For example, the email of a tutor would be included for every student they have.) If a student or tutor changed their email address it would have to be changed in every tuple where it occurred.

A similar problem would arise with the scheme **BARCODEDITEM**. In that case, several different barcodes may refer to the same product. So there may be several tuples relating to the same product. If the store wants to change the price it charges for that product, all the tuples for that product must be located and changed.

See page 15.

<i>barcode</i>	<i>product</i>	<i>cost</i>
53151	400 g mature cheddar	299
77502	Own brand Wheatflakes, Large	99
77505	Own brand Wheatflakes, Large	99
77507	Own brand Wheatflakes, Large	99

To avoid such problems, we seek to use schemes that are in **normal form**. There are a number of normal forms, and we shall consider first, second and third normal forms. We shall seek to divide a scheme into a collection of relation schemes, each of which is in third normal form. In doing this, we use the functional dependencies of the scheme.

The collection of schemes resulting from the division should form a relational database that avoids duplication of data and is straightforward to maintain. Although we will develop a strategy for doing this, you should note that there is not a unique “correct” database for any particular collection of data. It is not appropriate in this course to discuss details of database design; the intention here is to discuss some mathematical ideas underlying such design.

Second and third normal forms can be defined formally in terms of the functional dependencies of the scheme, and we shall turn to these shortly. We first deal rather briefly with first normal form. This relates to the choice of attributes and types used in defining the scheme. The problems we wish to avoid are illustrated in the table below.

	<i>commonName</i>	<i>site</i>	<i>season</i>
s1	Dipper	Lathkill Dale	Spring, Summer
s2	Dipper	Bradford Dale	Spring
s3	Kingfisher	Tim’s garden	Summer, Winter
s4	Blackbird	Our garden	Spring
s5	Dipper	Bradford Dale	Spring

At first glance this seems quite reasonable, but as a table for a relation scheme it has problems. First, we have duplicate rows (s2 and s5), easy enough to generate when collecting data but not wanted in a relation. Second, s1 and s3 present a problem in the *season* column. This is because double entries have been made. They would be awkward to work with if we wanted to extract all the tuples relating to Summer, say, or to display the tuples according to season. First normal form is intended to rule out such tables.

We say that a table is in **first normal form** if it has no duplicate rows and a single-valued entry in each cell.

In the scheme **SITE** given on page 28, s1 and s3 above are not valid tuples. The domain of the attribute *season* excludes such “multiple values”. First normal form can be seen as a caution about sensible domains to choose for the attributes of a scheme. You should not store in an attribute data that you will want to “divide” into separate items. Aside from the example above, all the tables in this unit are in first normal form. A table in first normal form can always be described as a relation over a relation scheme.

4.1 Second normal form

The following example illustrates a feature that a scheme may have, and which will lead to problems with updating entries.

Activity 4.1

Consider the following relation scheme **STUDENTANDCOURSE** that one might find in a university database. (A student may study more than one course, and more than one student may share a postcode.)

STUDENTANDCOURSE			Relation scheme
Attribute	Domain	Type	
<i>studentID</i>	<i>StudentIDCode</i>	mString	
<i>postcode</i>	<i>Postcode</i>	Postcode	
<i>courseCode</i>	<i>CourseCode</i>	mString	
Functional dependencies			
FD1: { <i>studentID</i> } $\rightsquigarrow$ { <i>postcode</i> }			

For convenience in this section, we will often start with a scheme whose functional dependencies are given. We saw in Section 2 how the functional dependencies may be deduced from given rules about a scheme.

Give a key of this scheme. Can you see any problems with this scheme?

Discussion The only key is $K = \{studentID, courseCode\}$. FD1 ensures that this has the determining property. A student may do several courses (and a course will have several students), so $\{studentID\} \not\rightsquigarrow \{courseCode\}$ and $\{courseCode\} \not\rightsquigarrow \{studentID\}$. Hence K also has the minimal property.

A problem with this scheme is that the student’s postcode is recorded for every course studied. This data duplication can cause a variety of problems. In particular, should a student change address, updating will be tricky.

In this example, the relation stores duplicated data as a result of a feature of the scheme. We have an attribute, *postcode*, that is determined by part of the key. We only need to know the *studentID* to know *postcode*.

A scheme in second normal form, defined below, avoids such problems. The term “proper” is a crucial part of this definition. The set *P* in the definition is part of the key *K*, not the whole key.

Recall that a proper subset of a set *K* is a subset of *K* that is not the whole set *K*.

We say that a relation scheme is in **second normal form** if we can **not** find the following:
a key *K*, an attribute *x* that is not in *K*, and a proper subset *P* of *K*, with $P \rightsquigarrow \{x\}$.

If you have a relation scheme that is not in second normal form you can split it into relation schemes that are in second normal form. We first illustrate how this can be done for **STUDENTANDCOURSE**.

EXAMPLE 4.1

In **STUDENTANDCOURSE**, $\{studentID, courseCode\}$ is a key. Now $\{studentID\}$ is a proper subset of this key and $\{studentID\} \rightsquigarrow \{postcode\}$. Hence this scheme is not in second normal form. To divide the scheme **STUDENTANDCOURSE** into schemes that are in second normal form we first form a new scheme from the attribute $\{studentID\}$ together with any attributes functionally dependent on it. This gives a scheme with attributes *studentID* and *postcode*, which we will call **STUDENT_DETAILS**. We also form a second scheme containing the original key together with any remaining attributes. In this case the attributes of the second scheme are just those in the original key: *studentID* and *courseCode*. We will call this second scheme **STUDENT_COURSE**.

We can summarise the two schemes as below. We need to give any functional dependencies that each scheme has. (As it happens, the first has just one explicit dependency, and the second has none.)

We have already given the type and domain of the attributes of these schemes, so need not repeat them.

STUDENT_DETAILS

Attributes: *studentID*, *postcode*
Functional dependencies:
FD1: $\{studentID\} \rightsquigarrow \{postcode\}$

STUDENT_COURSE

Attributes: *studentID*, *courseCode*

These two schemes are both in second normal form. There is a one to many association between **STUDENT_DETAILS** and **STUDENT_COURSE**, and $\{studentID\}$ is a foreign key in **STUDENT_COURSE**.

The general strategy for dividing a scheme into schemes in second normal form is given below.

Strategy 4.1

How to move to second normal form

Since the scheme is not in second normal form there will be a set of attributes P that is a proper subset of a key K and that functionally determines some attributes not in K .

Divide the scheme into two schemes.

- The attributes in P together with any attributes that are functionally dependent on P .
- K together with any attributes outside K that are not determined by P .

If either of these schemes is not in second normal form then repeat the process. Continue until all the schemes are in second normal form.

As with the example above, the schemes resulting from the division of the original scheme will have some attributes in common. This is necessary, otherwise we could not connect the schemes using a foreign key.

This includes both attributes that are explicitly dependent on P and any attributes that are implicitly dependent on P .

It will not always be the case that the schemes resulting from the first division are in second normal form. The first division will eliminate at least one case where a subset of a key determines attributes outside the key, but may not eliminate all such occurrences. If this is the case, we need to repeat the splitting process until all such occurrences are eliminated.

Activity 4.2

A relation scheme **VET** is given below, together with a relation over it. Is **VET** in second normal form? If not, divide it into schemes that are in second normal form.

VET			Relation scheme
Attribute	Domain	Type	
<i>petName</i>	<i>PetName</i>	mString	
<i>petType</i>	<i>Animal</i>	mString	
<i>age</i>	<i>Int</i>	Int	
<i>ownerName</i>	<i>Name</i>	mString	
<i>ownerID</i>	<i>ownerIDCode</i>	mString	
Functional dependencies			
FD1: { <i>petName</i> , <i>ownerID</i> } $\rightsquigarrow$ { <i>petType</i> , <i>age</i> , <i>ownerName</i> }			
FD2: { <i>ownerID</i> } $\rightsquigarrow$ { <i>ownerName</i> }			

The *ownerID* is a unique identifier.

We assume that no owner has two different pets with the same name.

	<i>petName</i>	<i>petType</i>	<i>age</i>	<i>ownerName</i>	<i>ownerID</i>
t1	Fido	dog	5	G.Brown	234
t2	Tiddles	cat	4	J.Jones	123
t3	Fido	dog	2	J.Jones	123
t4	Rover	dog	2	J.Jones	123
t5	Fido	dog	5	G.Brown	345

Notice that there are two owners named G. Brown, who are distinguished from each other by having different *ownerID* codes.

Discussion FD1 shows that $\{petName, ownerID\}$ determines all the other attributes. The tuples t1 and t3 show that $\{petName\}$ does not have the determining property (for example, it does not determine $ownerID$), and tuples t2 and t3 show that $\{ownerID\}$ does not determine $petName$.

Thus $\{petName, ownerID\}$ satisfies both the determining property and the minimal property, and so is a key of **VET**.

We are given (FD2) that $\{ownerID\} \rightsquigarrow \{ownerName\}$ is a dependency of **VET**. Since $\{ownerID\}$ is a proper subset of a key to the scheme, we can see that **VET** is not in second normal form.

We divide the scheme using Strategy 4.1. First, form a scheme with the attribute $ownerID$ together with any attributes dependent on it, in this case $ownerName$. Second, form a scheme whose attributes are the key, together with any attributes of **VET** not determined by $\{ownerID\}$. This divides **VET** into two schemes, which we shall call **PET_OWNER** and **PET**.

PET_OWNER

Attributes: $ownerID, ownerName$

Functional dependencies

FD2: $\{ownerID\} \rightsquigarrow \{ownerName\}$

PET

Attributes: $ownerID, petName, petType, age$

Functional dependencies

FD1: $\{petName, ownerID\} \rightsquigarrow \{petType, age\}$

These schemes are in second normal form (see below). **PET_OWNER** has a one to many association with **PET**, and $\{ownerID\}$ is the foreign key.

Notice that when we divide the scheme **VET**, its dependency FD2 becomes part of **PET_OWNER**, while FD1 becomes part of **PET**. However, we must remove $ownerName$ from the right of FD1, since that attribute is not part of **PET**.

We can divide the particular relation given in Activity 4.2 into associated tables over the two schemes as below. Notice how the Pet Table has one tuple for each tuple in the original table, but the PetOwner Table has fewer.

PetOwner Table		Pet Table				
<i>ownerID</i>	<i>ownerName</i>	<i>ownerID</i>	<i>petName</i>	<i>petType</i>	<i>age</i>	
123	J.Jones	123	Tiddles	cat	4	t2
		123	Fido	dog	2	t3
		123	Rover	dog	2	t4
234	G.Brown	234	Fido	dog	5	t1
345	G.Brown	345	Fido	dog	5	t5

To be certain that the subdivision in Activity 4.2 is complete, we should verify that **PET_OWNER** and **PET** are both in second normal form. To fail the test for second normal form, we need to have a key with a proper subset, and at least one attribute outside the key — which means there must be at least three attributes. The scheme **PET_OWNER** has only two attributes, and so must be in second normal form, since there simply are not enough attributes to fail the test.

Activity 4.3

Show that the scheme **PET**, summarised below, is in second normal form.

PET

Attributes: $ownerID, petName, petType, age$

Functional dependencies

FD1: $\{petName, ownerID\} \rightsquigarrow \{petType, age\}$

Discussion $\{petName, ownerID\}$ is a key of this scheme, for the same reasons as were given when showing that it is a key of **VET** in the previous activity. It is the only key, as is shown below. We can then see that the scheme is in second normal form, since neither $petName$ nor $ownerID$ determine any other attributes.

The tuples t_3 and t_4 show that $\{petType, age, ownerID\}$ does not determine $petName$, while the tuples t_1 and t_5 show that $\{petName, petType, age\}$ does not determine $ownerID$. This shows that any set of attributes with the determining property must contain both $petName$ and $ownerID$. Consequently, $\{petName, ownerID\}$ is the only set of attributes with both the determining and minimal properties, and so is the only key to the scheme.

There could be a cat called Fido or a goldfish called Rover.

Activity 4.4

Is the scheme **BARCODEDITEM** in second normal form?

See page 16.

Discussion Yes. The only key of **BARCODEDITEM** is $\{barcode\}$. Since the key only has one attribute, it has no proper subsets that could determine an attribute outside the key. So the condition for second normal form must hold.

4.2 Third normal form

We noted at the start of this section that there may be updating problems associated with the scheme **BARCODEDITEM**. This updating problem is not solved by second normal form — this scheme is in second normal form! Here, we develop a further normal form that helps us to identify and deal with such problems.

This scheme has two functional dependencies with a particular form:

$\{barcode\} \rightsquigarrow \{product\}$ and $\{product\} \rightsquigarrow \{cost\}$.

It is this situation that leads to the updating problem here. We call this linked pair of dependencies a **transitive dependency**, and say that $cost$ is **transitively dependent** on $\{barcode\}$. The first step towards defining third normal form is to define this idea in a way that is sufficiently general.

In a relation scheme we say that an attribute a is **transitively dependent** on a set of attributes X , where X is a key of the scheme, if there is another set of attributes Y such that

$$X \rightsquigarrow Y \text{ and } Y \rightsquigarrow \{a\} \tag{4.1}$$

but

$$Y \not\rightsquigarrow X \text{ and } a \notin X \cup Y. \tag{4.2}$$

Transitive dependency is related to derivation by transitivity. However, dependencies derived by transitivity may not satisfy condition (4.2).

The second condition here requires that the transitively dependent attribute is not involved in the first dependency. In the example above, the attribute $cost$ does not appear in the first dependency, $\{barcode\} \rightsquigarrow \{product\}$. Condition (4.2) also requires that the first dependency cannot be reversed. Here, $\{product\} \not\rightsquigarrow \{barcode\}$. (If $barcode$ were determined by $product$, there could be only one barcode for each product, and so there would be no updating problem in his example.)

We say that a relation scheme is in **third normal form** if it is in second normal form and has no transitive dependencies.

If a relation scheme is in second normal form but not in third normal form, then you can split it into schemes that are in third normal form by placing the transitively dependent attribute a in a separate scheme. If the original scheme is complicated, we need to check a couple of things before we do this: that the attribute a does not determine anything itself, and that we have found the smallest subset that determines a . We may also need to repeat the process, until all the transitive dependencies have been removed.

Strategy 4.2

How to move to third normal form

If the scheme is not in third normal form then there will be a transitive dependency $X \rightsquigarrow Y$ and $Y \rightsquigarrow \{a\}$, and where a does not itself determine any other attribute (see note 2). Identify the smallest subset (say B) of Y that determines a (see note 3). Form two schemes as follows.

- All the attributes except a .
- The set of attributes B with the attribute a added.

Repeat this process until there are no further transitive dependencies.

Notes on Strategy 4.2

1. One should first check that the scheme is in second normal form.
2. To be precise, we should add “except perhaps for an attribute which determines a ”. Any such attribute should be included with B and a when the scheme is divided.
3. The need to use B rather than the whole of Y is illustrated in the next activity.

This strategy would divide **BARCODEITEM** into the two schemes $\{barcode, product\}$ and $\{product, cost\}$, where $\{product\}$ in the first scheme is the foreign key. These schemes are in third normal form, and this division removes the updating problem discussed earlier. Now, if we change the price of a product, we need to change only one tuple over the second scheme.

Activity 4.5

The scheme **CUSTOMER** is used by a store to mail promotional material to customers who have a store card. There may be two different card holders living at the same address and with the same name. Some mailings are different for different areas of the country, so each address is assigned to a region. You may assume that this scheme is in second normal form. Is it in third normal form? If not, show how to divide it into schemes that are in third normal form.

CUSTOMER

Relation scheme

Attribute	Domain	Type
<i>cardNo</i>	<i>CardNumber</i>	Int
<i>name</i>	<i>Name</i>	mString
<i>address</i>	<i>Address</i>	mString
<i>region</i>	<i>RegionCode</i>	Int

Functional dependencies

- FD1: $\{cardNo\} \rightsquigarrow \{name, address\}$
FD2: $\{address\} \rightsquigarrow \{region\}$

Discussion From FD2, we can deduce the dependency $\{name, address\} \rightsquigarrow \{region\}$. Then we have the following pair of dependencies, in the form of a transitive dependency (4.1).

$\{cardNo\} \rightsquigarrow \{name, address\}$ and $\{name, address\} \rightsquigarrow \{region\}$.

Since there may be two different card holders living at the same address and with the same name, $\{name, address\} \not\rightsquigarrow \{cardNo\}$. Also, *region* is in neither of the sets $\{cardNo\}$ or $\{name, address\}$. So condition (4.2) also holds, and this is a transitive dependency.

Using Strategy 4.2, we divide the scheme as follows.

A scheme **CUSTOMER_A**, with attributes *cardNo*, *name* and *address*, and the functional dependency FD1 from **CUSTOMER**.

A scheme **REGION**, with attributes *address* and *region*, and the functional dependency FD2 from **CUSTOMER**.

Note that *region* is determined by the single attribute *address* (FD2). So the set *B* in Strategy 4.2 is $\{address\}$ in this case.

Each of these schemes is in third normal form. There is a one to many association between **REGION** and **CUSTOMER_A**, and *address* is a foreign key in **CUSTOMER_A**.

EXAMPLE 4.2

The relation scheme **BUS** was described in Example 2.1, and has the following functional dependencies.

FD1: $\{busCode, day\} \rightsquigarrow \{driverID\}$.

FD2: $\{driverID, day\} \rightsquigarrow \{busCode\}$.

FD3: $\{busCode, day\} \rightsquigarrow \{routeCode\}$.

There are two keys for this scheme — $\{busCode, day\}$ and $\{driverID, day\}$. See Activity 2.4.

No proper subset of either of these keys determines any attributes outside the key (because none of *busCode*, *driverID* or *day* on its own determines any other attribute), so **BUS** is in second normal form.

There are three pairs of dependencies that satisfy condition (4.1).

$\{busCode, day\} \rightsquigarrow \{driverID, day\}$ and $\{driverID, day\} \rightsquigarrow \{busCode\}$.

$\{driverID, day\} \rightsquigarrow \{busCode, day\}$ and $\{busCode, day\} \rightsquigarrow \{routeCode\}$.

$\{driverID, day\} \rightsquigarrow \{busCode, day\}$ and $\{busCode, day\} \rightsquigarrow \{driverID\}$.

However, condition (4.2) is not satisfied by any of these, because, in each case, the first dependency of the pair also holds in reverse.

So none of these is a transitive dependency. In fact, **BUS** is in third normal form.

This example illustrates the need for condition (4.2) in the definition of a transitive dependency. There is no way the attributes of this scheme can be divided while preserving the integrity of the data.

The strategies described in this section will always lead to a division of the original scheme into third normal form. However, for some schemes, the resulting division may not be the most suitable. An alternative approach is to divide the original scheme as follows. Have one new scheme containing the attributes involved in each of the explicit dependencies, plus a scheme containing the attributes of a key. We shall not consider that approach here.

This may lead to an unnecessarily large number of schemes in the division, so we then determine which of the resulting schemes can be discarded or combined.

Activity 4.6

The relation scheme **HORSE_A**, below, modifies the scheme in Activity 2.5 to include further information. In this, *damsWin* gives the dam’s career winnings (in pounds), and *siresWin* gives the sire’s career winnings.

HORSE_A			Relation scheme
Attribute	Domain	Type	
<i>name</i>	<i>HorseName</i>	mString	
<i>dam</i>	<i>HorseName</i>	mString	
<i>sire</i>	<i>HorseName</i>	mString	
<i>damsWin</i>	<i>Int</i>	Int	
<i>siresWin</i>	<i>Int</i>	Int	
<i>birthYear</i>	<i>Year</i>	Int	
<i>gender</i>	{‘M’, ‘F’}	Char	
Rules			
R1: No two horses have the same name.			
R2: No female horse has more than one foal per year.			
R3: Each horse has exactly one dam, sire, gender and birth year.			
R4: The career winnings are a single value associated with a horse.			

The table below shows a sample relation over the scheme.

	<i>name</i>	<i>dam</i>	<i>sire</i>	<i>damsWin</i>	<i>siresWin</i>	<i>birthYear</i>	<i>gender</i>
t1	FlashHarry	FlashLady	Gordon	37 550	440 270	1995	M
t2	PrettyOne	FlashLady	FastBoy	37 550	275 500	1996	F
t3	SteadyGirl	WhiteHoof	FastBoy	12 720	275 500	1996	F
t4	Frisky	FlashLady	Gordon	37 550	440 270	1997	M

Write down the explicit functional dependencies of **HORSE_A**. Is **HORSE_A** in second normal form? (You may assume that its only keys are {*name*} and {*dam*, *birthYear*}). If it is not, then divide it into schemes that are in second normal form. If the new schemes are not in third normal form, then divide them into schemes that are. Describe any associations between the schemes that you have defined, stating what the foreign keys are.

Discussion The explicit functional dependencies are as follows.

- FD1: {*dam*, *birthYear*} $\rightsquigarrow$ {*name*} (from R2).
- FD2: {*name*} $\rightsquigarrow$ {*dam*, *sire*, *birthYear*, *gender*} (from R3).
- FD3: {*dam*} $\rightsquigarrow$ {*damsWin*} (from R4).
- FD4: {*sire*} $\rightsquigarrow$ {*siresWin*} (from R4).

FD3 shows that a proper subset of the key {*dam*, *birthYear*} determines an attribute not in that key. So **HORSE_A** is not in second normal form.

We divide the scheme using Strategy 4.1, into schemes **DAM** and **HORSE_B**. **DAM** has attributes *dam* and *damsWin*. (Note that *damsWin* is the only attribute determined by *dam*.) **HORSE_B** has all the attributes of **HORSE_A** except *damsWin*. These schemes are summarised below.

DAM

Attributes: *dam*, *damsWin*

Functional dependencies

FD3: $\{dam\} \rightsquigarrow \{damsWin\}$

HORSE_B

Attributes: *name*, *dam*, *sire*, *siresWin*, *birthYear*, *gender*

Functional dependencies

FD1: $\{dam, birthYear\} \rightsquigarrow \{name\}$

FD2: $\{name\} \rightsquigarrow \{dam, sire, birthYear, gender\}$

FD4: $\{sire\} \rightsquigarrow \{siresWin\}$

The keys of **HORSE_B** are $\{name\}$ and $\{dam, birthYear\}$. The only key for **DAM** is $\{dam\}$. Both schemes are now in second normal form, since neither of the attributes *dam* or *birthYear* determines any other attributes.

The scheme **HORSE_B** does have a transitive dependency. We have a pair of dependencies satisfying (4.1):

$\{name\} \rightsquigarrow \{sire\}$ and $\{sire\} \rightsquigarrow \{siresWin\}$.

(The first is a consequence of FD2 and the second is FD4.) We should check that these also satisfy condition (4.2). Different horses can have the same sire, as is illustrated by the tuples t1 and t4. So $\{sire\} \not\rightsquigarrow \{name\}$. Also, *siresWin* is not one of the attributes involved in the first dependency $\{name\} \rightsquigarrow \{sire\}$.

We divide **HORSE_B** using Strategy 4.2 into the two schemes below.

SIRE

Attributes: *sire*, *siresWin*

Functional dependencies

FD4: $\{sire\} \rightsquigarrow \{siresWin\}$

HORSE_C

Attributes: *name*, *dam*, *sire*, *birthYear*, *gender*

Functional dependencies

FD1: $\{dam, birthYear\} \rightsquigarrow \{name\}$

FD2: $\{name\} \rightsquigarrow \{dam, sire, birthYear, gender\}$

We have now divided **HORSE_A** into three schemes. These are now all in third normal form. The schemes **DAM** and **SIRE** are clearly in third normal form. Each has only two attributes, so there can be no transitive dependencies. The scheme **HORSE_C** does have a pair of dependencies that satisfy (4.1): $\{dam, birthYear\} \rightsquigarrow \{name\}$ and $\{name\} \rightsquigarrow \{sire\}$. However these do not form a transitive dependency since (4.2) is not satisfied. The first dependency in the pair can be reversed: $\{name\} \rightsquigarrow \{dam, birthYear\}$ also holds.

SIRE has a one to many association with **HORSE_C**, with foreign key $\{sire\}$. **DAM** has a one to many association with **HORSE_C**, with foreign key $\{dam\}$.

Notice how each functional dependency of the original scheme is associated with one of the two schemes formed in the division.

sire could be replaced by *gender* here, but (4.2) still does not hold!

A sire may have fathered several horses, and a dam may have had several foals.

Objectives for Section 4

After studying this section you should be able to do the following.

- ☐ Determine whether a scheme is in second or third normal form;
- ☐ Divide a given scheme into associated schemes all of which are in third normal form.

Exercises on Section 4

Exercise 4.1

The scheme **STUDENT_TUTOR** is summarised below. Determine all the keys of this scheme, and decide whether it is in second normal form. If it is not, then divide it into schemes that are in second normal form. If the new schemes are not in third normal form, then divide them into schemes that are. Describe any associations between the schemes that you obtain, stating what the foreign keys are.

See page 11 and Exercise 2.1.

STUDENT_TUTOR

Attributes: *courseCode*, *studentID*, *tutorID*, *studentEmail*, *tutorEmail*

Functional dependencies

FD1: {*tutorID*} $\rightsquigarrow$ {*tutorEmail*}

FD2: {*studentID*} $\rightsquigarrow$ {*studentEmail*}

FD3: {*studentID*, *courseCode*} $\rightsquigarrow$ {*tutorID*}

Exercise 4.2

A travel company Wet and Cold Trips (plc) uses a scheme **MAIL_LIST** to store information on potential customers. They send mailings of various types, stored as *mailType*, and *dateMail* gives the date of the next mailing to the customer. A sample relation over **MAIL_LIST** is also given below.

MAIL_LIST			Relation scheme
Attribute	Domain	Type	
<i>name</i>	<i>Name</i>	mString	
<i>address</i>	<i>Address</i>	mString	
<i>postcode</i>	<i>Postcode</i>	mString	
<i>enquiry</i>	<i>Bool</i>	Bool	
<i>mailType</i>	<i>MailType</i>	Int	
<i>dateMail</i>	<i>Date</i>	Date	
<i>Rules</i>			
R1: No two customers share both name and address.			
R2: A customer has only one <i>enquiry</i> status.			
R3: Each address has only one postcode.			
R4: The type of mailing is determined by <i>postcode</i> and <i>enquiry</i> .			
R5: The date of the next mailing depends only on the type of mailing.			

	<i>name</i>	<i>address</i>	<i>postcode</i>	<i>enquiry</i>	<i>mailType</i>	<i>dateMail</i>
t1	W.Tunncliffe	7 Main Street, Kinraig	PH21 2ZZ	false	2	12 June
t2	T.Lister	4 Dylan Road, Nefyn	LL53 1QQ	true	3	15 July
t3	C.Thomas	27 Iffley Lane, Oxford	OX2 4GG	false	1	31 May
t4	B.Gabrys	28 Iffley Lane, Oxford	OX2 4GG	true	4	12 June
t5	C.Thomas	5 Iffley Close, Oxford	OX2 4GG	false	1	31 May
t6	W.Tunncliffe	25 Crookes, Sheffield	S10 2KK	true	3	15 July
t7	H.Tunncliffe	25 Crookes, Sheffield	S10 2KK	true	3	15 July

Write down the explicit functional dependencies of **MAIL_LIST**. Determine all the keys of this scheme and decide whether it is in second normal form. If it is not, then divide it into schemes that are in second normal form. If the new schemes are not in third normal form, then divide them into schemes that are. Describe any associations between the schemes that you obtain, stating what the foreign keys are.

Exercise 4.3

A coach company keeps information about its fleet in a scheme **COACH**, summarised below. Determine all the keys of this scheme and decide whether it is in second normal form. If it is not, then divide it into schemes that are in second normal form. If the new schemes are not in third normal form, then divide them into schemes that are. Describe any associations between the schemes that you obtain, stating what the foreign keys are.

COACH

Attributes: *coachID*, *age*, *mileage*, *garage*, *lastMaintained*, *mechanic*, *maintenanceDue*

Functional dependencies

FD1: {*coachID*} $\rightsquigarrow$ {*age*, *mileage*, *garage*, *lastMaintained*, *mechanic*, *maintenanceDue*}

FD2: {*garage*, *lastMaintained*} $\rightsquigarrow$ {*mechanic*}

FD3: {*age*, *mileage*, *lastMaintained*} $\rightsquigarrow$ {*maintenanceDue*}

Summary of Unit 11

1. A relation scheme consists of a set of attributes, together with information about how these are connected. A relation over the scheme can be seen as a table, with one column headed by each attribute. The rows in such a table are called tuples.
2. Suppose that, whenever two tuples have the same values for all the attributes in a set X , then the value of another attribute a must also be the same in each tuple. We say that a is functionally dependent on X , and write $X \rightsquigarrow \{a\}$. Functional dependencies that follow from the given rules about a scheme are called explicit. There will be other functional dependencies, called implicit; some that always hold, and some that are a consequence of the explicit dependencies.

A key of a relation scheme is a set of attributes that picks out a unique tuple in any relation over the scheme. A key can be recognised from the functional dependencies: it will determine all the attributes of the scheme (the determining property), and no proper subset of the key will do so (the minimal property). A scheme may have more than one key.

3. A relational database consists of a number of associated schemes. These schemes will have some attributes in common. Two of these schemes may be in various forms of association: one to one, one to many, or many to many. Where there is a one to many (or one to one) association, the schemes can be linked by a foreign key.
4. Normal forms are introduced to avoid the use of tables with repeated data, which can lead to problems when updating a relation. They are defined through the functional dependencies of the scheme. Second normal form requires that no attribute is dependent on a proper subset of a key. Third normal form also requires that there are no transitive dependencies. We showed how a scheme may be divided into a group of associated schemes, all of which are in third normal form

Solutions to the exercises

Solution 1.1

(a) (Dipper, **false**, river banks) is the first. Any of the other given tuples would do. (The only tuples that you can be sure are in the relation are those given.)

(b) (i) and (iv) could be added. (ii) could not be added because “don’t know” is not a value in **Bool**. So this tuple gives a value to the attribute *migrant* that is not in the appropriate domain. (iii) could not be added because it would break R2. There is already an entry for Blue Tit, which gives a different value for *nestSite*.

To be sure that the other tuples are valid entries we need to know that the values of the attributes are in the domains. That is, we need to know that each of “Magpie” and “Skylark” is in *BirdName*, and that “trees” and “on ground” are in *NestSites*.

Solution 2.1

In Example 2.5, we gave the following dependencies for **STUDENT_TUTOR**.

FD1: $\{tutorID\} \rightsquigarrow \{tutorEmail\}$.

FD2: $\{studentID\} \rightsquigarrow \{studentEmail\}$.

FD3B: $\{studentID, courseCode\} \rightsquigarrow \{tutorID, tutorEmail, studentEmail\}$.

We see from FD3B that $\{studentID, courseCode\}$ has the determining property.

Now in Activity 2.3, we gave examples of pairs of tuples that could occur together in a relation over **STUDENT_TUTOR**, which show that

$\{studentID\} \not\rightsquigarrow \{courseCode\}$ and $\{courseCode\} \not\rightsquigarrow \{tutorID\}$.

So neither $\{studentID\}$ nor $\{courseCode\}$ has the determining property.

Consequently, $\{studentID, courseCode\}$ has the minimal property.

Hence $\{studentID, courseCode\}$ has both the determining and minimal properties, and so is a key of the scheme.

Solution 2.2

Each tuple records the details of an individual student (R1), and R2 requires that a student only gives one postcode and one email address. A student can only have one name and one value for *current*. So we have the functional dependency

FD1: $\{studentID\} \rightsquigarrow \{name, postcode, email, current\}$.

It is possible for there to be two students with the same name, living at the same address and with the same email address. (This is consistent with the given rules.)

So the tuples shown below could occur together in a relation.

t1: (M2345, Jones, Jones@pcnet.com, LS21 4CD, **true**)

t2: (S1234, Jones, Jones@pcnet.com, LS21 4CD, **true**)

This example shows that there are no other functional dependencies.

Let $K = \{studentID\}$. K has the determining property (from FD1). Since K has only one attribute, it also has the minimal property. So K is a key for the scheme.

Tuples t1 and t2 show that not even all the other attributes together determine *studentID*. So any key must contain this attribute. Hence K is the only key.

Solution 2.3

Since each room is used at most once on a given day (R1) we have the dependency

FD1: $\{date, room\} \rightsquigarrow \{tutor, courseCode\}$.

On a given day, a tutor can teach at most one course in at most one room (R2 and R3). So we have the dependency

FD2: $\{date, tutor\} \rightsquigarrow \{room, courseCode\}$.

Let $K1 = \{date, room\}$ and $K2 = \{date, tutor\}$. FD1 shows that $K1$ has the determining property, and FD2 shows that $K2$ has the determining property.

Sample tuples given in Activity 1.3 are reproduced below.

	<i>date</i>	<i>tutor</i>	<i>courseCode</i>	<i>room</i>
t1	March 1	Professor Plum	F123	Hlab
t2	March 1	Dr Apple	F245	LR24
t3	March 2	Ms Cherry	A104	LR21
t4	March 8	Dr Apple	F245	LR24

These show that no single attribute has the determining property. (t1 and t2 rule out $\{date\}$. t2 and t4 rule out $\{tutor\}$, $\{courseCode\}$ and $\{room\}$.) From this we can see that no proper subset of either $K1$ or $K2$ can have the determining property. So both these sets have the minimal property. We already showed that both have the determining property, so each of $K1$ and $K2$ is a key of this scheme.

Have we found all the keys? Tuples t2 and t4 show that any key must contain the attribute *date*. So the only possibility remaining is $\{date, courseCode\}$. The tuples above do not eliminate this possibility. But there is nothing in the rules of the scheme that would prevent there being two tutorials for the same course on the same day (with different tutors and different rooms). For example, the following tuple could occur with those above.

(March 2, Dr Singh, A104, LR24)

So $\{date, courseCode\}$ does not have the determining property, and $K1$ and $K2$ are the only keys of **CENTRE_TIMETABLE**.

Solution 3.1

For **STOCK**, the only dependency is $\{bookID\} \rightsquigarrow \{isbn\}$, and the only key is $\{bookID\}$. For **LOAN**, the only dependency is $\{bookID\} \rightsquigarrow \{inLibrary\}$, and the only key is again $\{bookID\}$. These schemes have a one to one association. The foreign key is $\{bookID\}$, but there is a choice of which scheme to locate it in — it is an alternate key.

The schemes **STOCK** and **BOOK** share the attribute *isbn*. The library may have several copies of the same publication (that is, several books with the same *isbn*), so a tuple in **BOOK** may have several corresponding tuples in **STOCK**. There is a one to many association between **BOOK** and **STOCK**. The foreign key is $\{isbn\}$ in **STOCK**.

Solution 3.2

TILL has $\{tillID\}$ as a key and **TRANSACTION** has $\{transID\}$ as a key. (The given functional dependencies show that these both have the determining property. Both have the minimal property since they each have only a single attribute.)

Each till will have many transactions, but each transaction can involve only one till. So **TILL** has a one to many association with **TRANSACTION**. The foreign key is $\{tillID\}$ in **TRANSACTION**.

Solution 4.1

$\{studentID, courseCode\}$ is a key for **STUDENT_TUTOR**. We showed that in Solution 2.1, and shall not repeat the explanation given there.

Now $\{studentID\} \rightsquigarrow \{studentEmail\}$ (FD2), so a proper subset of the key determines an attribute not in the key. This means the scheme is not in second normal form. We split the relation scheme using Strategy 4.1, as follows. First form a relation scheme from *studentID* and any dependent attributes.

STUDENT

Attributes: *studentID, studentEmail*

Functional dependencies

FD2: $\{studentID\} \rightsquigarrow \{studentEmail\}$

Then form a scheme from the rest of the attributes.

STUDENT_TUTOR_A

Attributes: *courseCode, studentID, tutorID, tutorEmail*

Functional dependencies

FD1: $\{tutorID\} \rightsquigarrow \{tutorEmail\}$

FD3: $\{studentID, courseCode\} \rightsquigarrow \{tutorID\}$

Both of these schemes are now in second normal form.

The scheme **STUDENT_TUTOR_A** has a transitive dependency. We have

$\{studentID, courseCode\} \rightsquigarrow \{tutorID\}$ and $\{tutorID\} \rightsquigarrow \{tutorEmail\}$,

while $\{tutorID\} \not\rightsquigarrow \{studentID, courseCode\}$

and *tutorEmail* is not in the set $\{studentID, courseCode, tutorID\}$.

Dividing **STUDENT_TUTOR_A** using Strategy 4.2 leads to two schemes. The first is formed by removing the attribute *tutorEmail*, and the second contains the attributes in the second part of the transitive dependency.

STUDENT_TUTOR_B

Attributes: *courseCode, studentID, tutorID*

Functional dependencies

FD3: $\{studentID, courseCode\} \rightsquigarrow \{tutorID\}$

TUTOR

Attributes: *tutorID, tutorEmail*

Functional dependencies

FD1: $\{tutorID\} \rightsquigarrow \{tutorEmail\}$

The original scheme has now been divided into three schemes, all in third normal form.

STUDENT is in a one to many association with **STUDENT_TUTOR_B**, with foreign key $\{studentID\}$.

A student may do several courses.

TUTOR is in a one to many association with **STUDENT_TUTOR_B**, with foreign key $\{tutorID\}$.

A tutor will have several students.

In this database, if a student or tutor changes their email address the details only need to be updated once.

Solution 4.2

The explicit functional dependencies that follow from the rules are given below.

FD1: $\{name, address\} \rightsquigarrow \{enquiry\}$ (from R1 and R2).

FD2: $\{address\} \rightsquigarrow \{postcode\}$ (from R3).

FD3: $\{enquiry, postcode\} \rightsquigarrow \{mailType\}$ (from R4).

FD4: $\{mailType\} \rightsquigarrow \{dateMail\}$ (from R5).

Let $K = \{name, address\}$. FD1 shows that K determines *enquiry*. FD2 shows that K determines *postcode*. Then FD3 shows that K determines *mailType*. Finally, FD4 then shows that K determines *dateMail*. Hence K has the determining property.

Now t1 and t6 show that $\{name\}$ does not determine *address*, while t6 and t7 show that $\{address\}$ does not determine *name*. Thus K satisfies the minimal property. Hence K is a key for the scheme.

The tuples t6 and t7 show that any key must contain *name*. Tuples t3 and t5 show that any key must contain *address*. This means that K is the only key of the relation scheme.

FD2 means that the scheme is not in second normal form, since a proper subset of the key, $\{address\}$, determines an attribute, *postcode*, that is not in the key. We use Strategy 4.1 to divide **MAIL_LIST** into two new schemes, as below. (In this case, the second scheme cannot “inherit” FD3 directly, since *postcode* has been removed from the scheme. But in the original scheme, FD3 leads to the implicit dependency $\{enquiry, address\} \rightsquigarrow \{mailType\}$, since *address* determines *postcode*.)

POSTCODE

Attributes: *address, postcode*

Functional dependencies

FD2: $\{address\} \rightsquigarrow \{postcode\}$

MAIL_LIST_A

Attributes: *name, address, enquiry, mailType, dateMail*

Functional dependencies

FD1: $\{name, address\} \rightsquigarrow \{enquiry\}$

FD3A: $\{enquiry, address\} \rightsquigarrow \{mailType\}$

FD4: $\{mailType\} \rightsquigarrow \{dateMail\}$

MAIL_LIST_A has $K = \{name, address\}$ as its only key for the same reasons as did **MAIL_LIST**. Since no subset of K determines any other attributes, this scheme is in second normal form. There are transitive dependencies, though. Condition (4.1) is satisfied by the dependencies

$\{enquiry, address\} \rightsquigarrow \{mailType\}$ and $\{mailType\} \rightsquigarrow \{dateMail\}$.

Also, (4.2) holds, since $\{mailType\} \not\rightsquigarrow \{enquiry, address\}$ (see t2 and t6 for example), and *dateMail* is not in $\{enquiry, address, mailType\}$. So **MAIL_LIST_A** is not in third normal form. Following Strategy 4.2, we divide **MAIL_LIST_A** into two new schemes.

MAILING

Attributes: *mailType, dateMail*

Functional dependencies

FD4: $\{mailType\} \rightsquigarrow \{dateMail\}$

MAIL_LIST_B

Attributes: *name, address, enquiry, mailType*

Functional dependencies

FD1: $\{name, address\} \rightsquigarrow \{enquiry\}$

FD3A: $\{enquiry, address\} \rightsquigarrow \{mailType\}$

We have now divided **MAIL_LIST** into three schemes. Of these, **POSTCODE** has $\{address\}$ as its only key, and **MAILING** has $\{mailType\}$ as its only key. Each of these has only two attributes, and so will be in third normal form.

However, we are still not finished. The scheme **MAIL_LIST_B** has a dependency $\{name, address\} \rightsquigarrow \{enquiry, address\}$. (FD1 shows that $\{name, address\}$ determines *enquiry*, and this set of attributes contains *address* and so is bound to determine it.) So we have the paired dependencies

$\{name, address\} \rightsquigarrow \{enquiry, address\}$ and $\{enquiry, address\} \rightsquigarrow \{mailType\}$.

Also, $\{enquiry, address\} \not\rightsquigarrow \{name, address\}$ (see tuples t6 and t7), and the attribute *mailType* does not appear in the first dependency. So (4.2) is also satisfied, and this is another transitive dependency. We need to use Strategy 4.2 to divide this scheme again.

MAIL_LIST_C

Attributes: *name, address, enquiry*

Functional dependencies

FD1: $\{name, address\} \rightsquigarrow \{enquiry\}$

MAIL_TYPE

Attributes: *address, enquiry, mailType*

Functional dependencies

FD3A: $\{enquiry, address\} \rightsquigarrow \{mailType\}$

Both of these schemes are now in third normal form. **MAIL_LIST_C** has $\{name, address\}$ as its only key. **MAIL_TYPE** has $\{enquiry, address\}$ as its only key.

POSTCODE has a one to many association with **MAIL_LIST_C**, with foreign key $\{address\}$. **MAIL_TYPE** has a one to many association with **MAIL_LIST_C**, with foreign key $\{enquiry, address\}$. **MAILING** has a one to many association with **MAIL_TYPE**, with foreign key $\{mailType\}$.

Incidentally, this example reveals some issues not covered by our discussion in this unit. The “real world” rule that lead to FD3 would associate a *mailType* with *enquiry* and *postcode* (rather than *address*). It would be natural to have a scheme with attributes *mailType*, *enquiry* and *postcode* in place of **MAIL_TYPE** as above. Had we designed a relational database with separate schemes directly, we might well have come up with such a design — but the rules given here for dividing a scheme that is not in third normal form may not lead to the most natural design.

Solution 4.3

We know from FD1 that $\{coachID\}$ has the determining property. Since this set contains a single attribute, it has the minimal property. Hence $\{coachID\}$ is a key for the relation scheme. No set of attributes determines *coachID* unless the set contains *coachID*. This means that $\{coachID\}$ is the only key to the scheme. The scheme is thus in second normal form as the only key has just one attribute.

This scheme is not in third normal form. We can see that there are two transitive dependencies, as below. (In each case, the first of the pair of dependencies is a consequence of FD1. Also, in each case the first dependency cannot be reversed, and does not involve the transitively dependent attribute, so (4.2) is also satisfied.)

$\{coachID\} \rightsquigarrow \{garage, lastMaintained\}$ and $\{garage, lastMaintained\} \rightsquigarrow \{mechanic\}$

$\{coachID\} \rightsquigarrow \{age, mileage, lastMaintained\}$ and $\{age, mileage, lastMaintained\} \rightsquigarrow \{maintenanceDue\}$

Using Strategy 4.2 twice, we split the scheme into three schemes as below.

COACH_A

Attributes: *coachID, age, mileage, garage, lastMaintained*

Functional dependencies

FD1: $\{coachID\} \rightsquigarrow \{age, mileage, garage, lastMaintained\}$

MECHANIC

Attributes: *garage, lastMaintained, mechanic*

Functional dependencies

FD2: $\{garage, lastMaintained\} \rightsquigarrow \{mechanic\}$

MAINTENANCE

Attributes: *age, mileage, lastMaintained, maintenanceDue*

Functional dependencies

FD3: $\{age, mileage, lastMaintained\} \rightsquigarrow \{maintenanceDue\}$

All three schemes are now in third normal form. **MAINTENANCE** has a one to many association with **COACH_A**, with foreign key $\{age, mileage, lastMaintained\}$. (It is just possible that there were two coaches of the same age and with the same mileage maintained on the same date.) **MECHANIC** has a one to many association with **COACH_A**, with foreign key $\{garage, lastMaintained\}$. It is possible that there were two coaches maintained on the same date by the same mechanic.

Index

- association
 - many to many, 28
 - one to many, 26
 - one to one, 29
- attribute, 4
 - domain of, 6
 - value of, 6
- data duplication, 29
- derivation by transitivity, 14
- determining property, 22
- domain
 - of an attribute, 6
- foreign key, 26
- functional dependency
 - explicit, 16
 - implicit, 16
 - trivial, 16
- functionally dependent, 12
- key
 - alternate, 29
 - candidate, 22
 - foreign, 26
 - primary, 22
- key of a relation scheme, 22
- minimal property, 22
- normal form
 - first, 32
 - second, 33
 - third, 37
- primary key, 22
- property
 - determining, 22
 - minimal, 22
- references
 - foreign key, 26
- relation, 5
- relation scheme, 7
- relational database, 25
- transitive dependency, 36
- tuple, 5